

PROJECT REPORT

A WEBSITE FOR EMAIL SENDING FOR BLIND PEOPLE USING SPEECH RECOGNITION TECHNOLOGY

Name: P. Babjan

4th Year

B.tech

Computer Science and Engineering

Registered Mail Id: palagiribabjan143@gmail.com

College Name: K.S.R.M COLLEGE OF ENGINEERING, KADAPA

Abstract:

- Blind and visually impaired individuals face significant challenges with traditional digital communication tools, especially email systems that lack accessibility features tailored to their needs.
- These systems often rely heavily on visual cues and manual navigation, limiting independence for users who depend on auditory feedback and voice commands.
- The platform enables users to dictate, compose, and send emails purely through voice commands, eliminating the need for visual interaction.

CHAPTER 1

Introduction

1-10

- 1.1 Introduction
- 1.2 Domain Description about the Area of Project
- 1.3 Problem Definition
- 1.4 Proposed Solution
- 1.5 Objective
- 1.6 Organization of the Project report

1. INTRODUCTION

1.1 Introduction

In today's digital era, email communication has become an essential part of both personal and professional life. From sending important documents to receiving critical updates, emails serve as a primary mode of communication across various domains. However, the widespread use of graphical user interfaces (GUIs) in traditional email platforms poses a significant barrier for visually impaired individuals. These systems are primarily designed for sighted users and often overlook the needs of those who rely on auditory or tactile feedback. Although screen readers offer some level of accessibility, they usually require keyboard navigation and can be cumbersome to use, particularly for individuals unfamiliar with complex command structures. Moreover, the lack of hands-free interaction, real-time voice command support, and smart text correction features make existing platforms inadequate for users who depend solely on voice for interaction. As a result, blind or visually impaired individuals often require assistance from others, reducing their independence and limiting their access to seamless digital communication.

To address these challenges, there is a growing need for an inclusive and voice-centric solution that allows visually impaired users to interact with email systems effectively and independently. Our project introduces a Voice-Controlled Email System specifically designed for blind users, offering a fully speech-driven interface that eliminates the reliance on visual elements. The platform is equipped with advanced speech recognition capabilities, enabling users to perform core functions such as registration, login, reading, composing, sending, replying, and deleting emails using voice commands alone. A custom-built mail server connected to a secure SQL database ensures smooth and reliable email operations. To enhance user experience, the system incorporates Natural Language Processing (NLP) techniques for intelligent text correction and Text-to-Speech (TTS) for reading messages aloud. Additionally, features such as voice-based authentication and logout mechanisms ensure data security and privacy.

1.2 Domain Description about the area of project

Machine Learning (ML) is a subfield of artificial intelligence (AI) that focuses on developing algorithms and statistical models that enable computer systems to perform specific tasks without using explicit instructions. Instead, these systems rely on patterns and inference derived from data. Machine learning is broadly categorized into supervised learning, unsupervised learning, semi-supervised learning, and reinforcement learning. Supervised learning involves labeled datasets to train models, such as in classification and regression tasks. Unsupervised learning, on the other hand, deals with unlabeled data and focuses on finding hidden patterns or intrinsic structures within the input. Reinforcement learning, inspired by behavioral psychology, teaches an agent to take actions in an environment to maximize cumulative reward. ML plays a pivotal role in applications such as fraud detection, recommendation systems, predictive maintenance, and image recognition.

Deep Learning (DL), a subset of machine learning, leverages artificial neural networks with multiple layers (hence “deep”) to model complex relationships in data. Deep learning excels particularly in scenarios involving large volumes of unstructured data like images, videos, and audio. Convolutional Neural Networks (CNNs) are commonly used in image-related tasks, while Recurrent Neural Networks (RNNs), and their more advanced form Long Short-Term Memory (LSTM) networks, are often utilized for sequential data like time-series and natural language. With the advent of more powerful hardware (such as GPUs and TPUs) and the availability of massive datasets, deep learning has led to groundbreaking results in fields like autonomous driving, medical imaging, and generative models. Deep learning models have also paved the way for creative AI, enabling systems to compose music, generate art, and even write stories. Natural Language Processing (NLP) is a specialized area of AI that deals with the interaction between computers and human (natural) languages. The goal of NLP is to enable machines to read, understand, interpret, and generate human languages in a valuable way.

Traditional NLP tasks include tokenization, stemming, lemmatization, part-of-speech tagging, named entity recognition and parsing. With the introduction of ML and DL techniques, NLP has evolved significantly, now encompassing complex tasks like sentiment analysis, machine translation, speech recognition, question answering, and summarization. The integration of Transformer-based models such as BERT, GPT, and T5 has transformed NLP by enabling context-aware language understanding and generation, thereby significantly boosting performance across a wide range of applications. The convergence of ML, DL, and NLP has resulted in powerful AI systems that can mimic human-like understanding and decision-making capabilities. For instance, chatbots and virtual assistants like Siri, Alexa, and Google Assistant utilize NLP to interpret user commands and ML/DL models to personalize responses and learn user preferences over time. In the healthcare sector, ML and DL models analyze patient data, radiology images, and genomic sequences, while NLP is used to interpret clinical notes and medical literature. In finance, algorithms detect fraudulent transactions, predict market trends, and automate customer service using conversational AI. The integration of these domains is revolutionizing not just technology, but also society at large by enabling intelligent automation and enhanced decision support systems.

From a research and development perspective, ML, DL, and NLP continue to evolve rapidly, driven by open-source frameworks such as TensorFlow, PyTorch, Keras, and Hugging Face Transformers. These tools have democratized access to state-of-the-art models and made it easier for researchers and developers to build, test, and deploy intelligent systems. Pretrained models and transfer learning have significantly reduced the time and data required to develop high-performing applications. Meanwhile, interpretability and ethical AI have become major areas of focus, as the widespread deployment of ML and DL systems raises questions about bias, transparency, fairness, and accountability.

1.3 Problem Definition

In a world where digital communication has become an integral part of everyday life, visually impaired individuals continue to face significant obstacles in accessing essential services like email. Email is a critical tool used in education, employment, healthcare, banking, and social interaction. However, most popular email platforms are designed with visual interaction in mind, heavily relying on graphical interfaces and mouse-based navigation. This design framework creates a digital divide that excludes those who cannot perceive visual content, making it extremely challenging for blind users to interact with email systems independently. The inability to efficiently send or read emails without assistance often leads to feelings of frustration, isolation, and dependency.

Although screen readers and assistive technologies have made commendable progress in offering accessibility, they still fall short in delivering a seamless and intuitive user experience. These tools generally require users to memorize complex keyboard shortcuts and navigate through cluttered interfaces filled with ads, buttons, and menus that are not optimized for auditory interpretation. Moreover, screen readers lack natural integration with features like real-time speech input, automated text correction, and quick voice commands, resulting in slow and inefficient communication. For many visually impaired users, even basic tasks like composing an email or reading a message thread can become tedious, time-consuming, and mentally exhausting. As a result, many users avoid using email altogether unless aided by another person, further reducing their digital independence.

In professional settings, this lack of accessibility poses a more serious concern. Many visually impaired individuals struggle to manage job-related emails on their own, affecting productivity and limiting career advancement opportunities. Employers may not always provide customized tools, and the reliance on standard interfaces that are not voice-friendly places visually impaired employees at a disadvantage. In educational institutions, blind students find it difficult to keep up with assignments, notifications, and communications from teachers that are typically sent via email. This affects their academic performance and confidence, creating yet another layer of inequality in an already competitive environment. The issue is not just technical but deeply social, excluding a portion of the population from something as fundamental as email communication impacts their overall inclusion in society.

Furthermore, the absence of user-friendly, voice-controlled email platforms exacerbates the daily challenges faced by visually impaired individuals in their personal lives. Whether it is applying for a job, receiving health-related updates, managing bills, or simply staying in touch with family and friends, email is a necessity. When blind users cannot access these services independently, it affects their mental well-being, limits their freedom, and undermines their ability to lead a self-sufficient life. In many parts of the world, especially in rural or underserved areas, the situation is even worse due to lack of awareness, technological support, or accessibility advocacy. This clearly illustrates the urgent need for inclusive technological solutions that address these critical gaps and support equal participation of visually impaired individuals in the digital age.

1.4 Proposed Solution

To bridge the digital divide and offer visually impaired individuals greater autonomy in communication, we propose the development of a Voice-Controlled Email System that is fully operable through speech commands. The core idea behind this solution is to eliminate the dependency on graphical interfaces by creating a voice-first environment that understands, processes, and responds to user commands in a natural and intuitive manner. This system leverages Speech Recognition Technology to allow users to perform all necessary email functions including account creation, login, composing, reading, sending, replying to, deleting, and navigating through emails without ever needing to see the screen. The primary user interaction will be entirely voice-driven, allowing users to speak their intentions clearly and receive audio feedback in return, thus making email usage not only accessible but also efficient and empowering for blind users.

A key feature of the proposed system is the integration of Natural Language Processing (NLP), which enhances the user's voice input by performing real-time text correction and grammar checking. When users dictate emails, the system processes the speech input to generate text that is both accurate and coherent. This eliminates common frustrations such as misinterpreted words or unclear sentence structures, which are often experienced when using basic voice-to-text systems. By employing NLP techniques, the system ensures that dictated emails are professional and readable, improving the overall communication quality. In addition, this feature provides a smoother user experience by reducing the need for manual correction or repeated inputs, which can be a challenge for users relying solely on voice.

To ensure that users can also consume content effectively, the system includes Text-to-Speech (TTS) technology that reads email content aloud in a natural and understandable tone. This enables visually impaired users to listen to emails at their convenience, replay important messages, and navigate through their inbox using simple voice commands. Each function whether it's opening a specific email, checking the sender, or reading the subject line is guided by speech, making the process fully interactive and non-visual. Furthermore, the system can be personalized to accommodate different speech speeds, voices, and languages, catering to the diverse preferences and needs of the visually impaired community.

Security and user privacy are also central to the proposed solution. The system incorporates voice-based authentication mechanisms for secure login and logout, ensuring that only authorized users can access personal information. This method is both accessible and reliable, replacing traditional password-based logins, which can be difficult for blind users to input without sighted assistance. Additionally, the system is supported by a custom-built mail server and a structured SQL database that manages user accounts, email data, and session tracking efficiently and securely. Together, these components create a holistic platform that not only solves the immediate communication barriers for visually impaired users but also fosters a greater sense of independence, confidence, and inclusion in the digital world.

1.5 Objectives

- To develop a fully voice-driven email system that enables visually impaired users to interact without relying on a graphical user interface.
- To integrate speech recognition technology for capturing and understanding user commands for various email operations like login, read, compose, send, and delete.
- To implement Natural Language Processing (NLP) techniques for accurate voice-to-text transcription and automatic text correction during email composition.
- To include Text-to-Speech (TTS) functionality that reads emails aloud, enabling users to listen to the content of messages easily and comfortably.
- To design a custom email server and database (SQL) that securely stores user data and manages email transactions efficiently.
- To ensure data privacy and security through voice-based authentication and logout mechanisms, removing the need for typing passwords.
- To simplify email navigation using intuitive voice commands, allowing users to move through inboxes, sent items, and other folders with ease.
- To reduce dependency on screen readers by providing a complete, hands-free alternative tailored specifically for the blind community.
- To promote digital inclusion and independence for visually impaired users by creating an accessible and empowering communication platform.
- To create a scalable and user-friendly system that can be adapted to multiple languages and regional dialects to serve a broader audience.

1.6 Organization of the Project report

The organization of a project report follows a structured format to ensure clarity, coherence, and a logical flow of information from the initial concept to the final outcomes. It typically begins with preliminary pages such as the cover page, declaration, certificate, acknowledgments, and table of contents. These are followed by the abstract, which offers a concise summary of the project, including the objective, methodology, key findings, and conclusion. The introduction chapter sets the context by describing the background, motivation, objectives, and scope of the project. It also outlines the problem statement and the significance of the study. The literature review section explores existing work related to the project topic, identifying research gaps and establishing the foundation upon which the proposed work is built.

The core chapters of the report present the methodology, which explains the techniques, tools, datasets and algorithms used in the project, along with system design and implementation details. This is followed by the experimental results and analysis, where the performance of the developed models or systems is evaluated using appropriate metrics, visualizations, and comparisons. The discussion section interprets the results and highlights key insights. The report concludes with the conclusion and future work, summarizing the achievements, limitations, and potential directions for further research or development. Finally, references and appendices are included to credit sources and provide supplementary information such as code snippets, diagrams or raw data. This systematic organization ensures the report is academically rigorous, easy to navigate and informative for both technical and non-technical readers.

CHAPTER 2

Literature Survey

11-19

- 2.1. Introduction
- 2.2 Related Work
- 2.3. Overview

2. LITERATURE SURVEY

2.1 Introduction

Over the years, several researchers and developers have explored ways to enhance digital accessibility for visually impaired individuals, particularly in the realm of email communication and voice-assisted technology. Traditional email systems like Gmail and Outlook offer limited support through screen readers such as JAWS and NVDA, but these require a steep learning curve and do not provide a fully voice-driven interface. Some efforts have been made to integrate basic voice input features into mobile platforms, but these solutions often lack precision, multilingual support, and deep contextual understanding, making them unreliable for professional or academic use. Early prototypes of voice-based applications have attempted to allow message dictation and retrieval, but they either depended heavily on internet connectivity or lacked robust backend systems to manage data securely. Furthermore, the absence of Natural Language Processing (NLP) in these applications led to a high rate of transcription errors, making the email content unprofessional or hard to understand.

In recent studies, the focus has shifted towards combining Artificial Intelligence (AI) with accessibility solutions to provide more intelligent, adaptive interfaces. Projects like VoiceMail, Talking Email, and Speak2Mail experimented with speech interfaces, but their limited functionality and lack of support for complex commands restricted their adoption. Few initiatives explored the inclusion of Text-to-Speech (TTS) for reading emails aloud, but these were not integrated into a complete email management system and often lacked user authentication features. Moreover, many of these earlier systems were not user-friendly or scalable, failing to serve a diverse and growing visually impaired population. The need for a comprehensive, secure, and fully voice-operated email system still remains largely unmet, setting the stage for modern, inclusive solutions that leverage advanced speech recognition, NLP, and TTS technologies in unison to revolutionize accessibility.

2.2 Related Work

- [1] Shah, Harsh D., Amit Sundas, and Shabnam Sharma. "Controlling email system using audio with speech recognition and text to speech." 2021 9th international conference on reliability, infocom technologies and optimization (trends and future directions) (ICRITO). IEEE, 2021.

The paper by Harsh D. Shah, Amit Sundas, and Shabnam Sharma, titled "Controlling email system using audio with speech recognition and text to speech", presented at the 2021 9th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO), explores the development of a voice-controlled email management system that enhances accessibility and user convenience. The authors propose a framework that allows users to perform email-related tasks such as reading, composing, sending, and managing emails through spoken commands using speech recognition technology. The system leverages automatic speech recognition (ASR) for converting spoken input into text and employs text-to-speech (TTS) for converting textual responses back into speech.

The architecture integrates key technologies including natural language processing (NLP), Google's speech recognition APIs, and Python-based scripting to ensure a responsive and user-friendly environment. The system workflow is designed such that after user authentication, verbal commands are parsed and converted to corresponding email actions, such as reading unread messages, composing new mails through dictation, or navigating folders. TTS is used to provide auditory feedback to the user, confirming actions or reading out message content. The authors highlight the system's potential in scenarios requiring multitasking or accessibility enhancement, especially for the visually impaired. While the project demonstrates effective integration of ASR and TTS technologies, the paper also acknowledges challenges such as accent variation, background noise interference, and the need for improved NLP understanding, suggesting areas for further refinement and robustness in future implementations.

[2] Umbare, Mrs Rupali, et al. "Voice Controlled Email System Using Speech Recognition."

The paper titled "Voice Controlled Email System Using Speech Recognition" by Mrs. Rupali Umbare and her co-authors presents a voice-interactive email system designed to facilitate email communication through speech commands. The primary objective of this work is to assist users especially those who are visually impaired or physically challenged in accessing, composing, and managing their emails without relying on traditional keyboard and mouse input. The system utilizes speech recognition to interpret user commands and convert spoken words into text, which is then processed to carry out corresponding email functions such as reading inbox messages, composing and sending emails, or navigating through folders. This voice-based interaction offers a hands-free solution, significantly enhancing accessibility and ease of use for users who may struggle with conventional interfaces.

The system architecture incorporates a combination of speech-to-text (STT) and text-to-speech (TTS) technologies, allowing users to issue commands and receive audio feedback in real time. The development relies on Python and various open-source APIs for handling speech input and email protocol functions (like SMTP for sending and IMAP for receiving messages). Upon launching the system, users are guided through voice prompts, enabling a conversational interface that minimizes the need for visual engagement. The paper emphasizes the practical applicability of the system in real-world contexts where users may need to operate email services in hands-busy environments or where screen-based interfaces are inaccessible. Although the solution proves functional and user-friendly, the authors acknowledge areas for enhancement, including more accurate voice recognition in noisy settings and improved command interpretation to handle a broader range of natural language expressions.

[3] Umbare, Rupali, et al. "AI Based Email System using Speech Recognition."

The paper titled “AI Based Email System using Speech Recognition” by Rupali Umbare and colleagues introduces an intelligent voice-controlled email system that integrates artificial intelligence with speech recognition to streamline email communication. Aimed particularly at visually impaired users and those with physical disabilities, the system replaces traditional input methods with speech commands, allowing users to compose, send, read, and manage emails using only their voice. It uses advanced speech-to-text (STT) algorithms to convert spoken input into actionable text, which is then processed by an AI engine to interpret the user’s intent. This intelligent layer of processing enables the system to execute commands more accurately, understand contextual phrases, and reduce the dependency on rigid keyword-based instructions.

In addition to its core STT functionality, the system incorporates text-to-speech (TTS) capabilities to provide auditory feedback to the user, effectively creating a bidirectional voice interface. The use of AI enhances the system’s ability to learn from user interactions and improve over time, offering a more adaptive and personalized experience. Implemented using Python and integrated with APIs for email services, the system supports basic functions like reading inbox messages and composing emails, as well as more advanced tasks such as managing folders or searching specific content. The paper highlights the effectiveness of this AI-based solution in making email communication more inclusive and efficient, while also identifying challenges such as speech recognition errors in noisy environments and the need for multilingual support as future areas for development.

[4] Chimkode, Suresh, and Sara Fatima. "Voice Based E-Mail System For Blind People Using Speech Recognition Technology."

The paper "Voice Based E-Mail System For Blind People Using Speech Recognition Technology" by Suresh Chimkode and Sara Fatima focuses on developing an email system that allows visually impaired individuals to interact with email services using only their voice. The proposed system leverages speech recognition technology to enable users to perform essential email tasks, such as composing, reading, sending, and organizing emails, through spoken commands. By eliminating the need for visual interaction, the system aims to provide greater independence and accessibility for blind users. The speech-to-text (STT) functionality converts the user's verbal instructions into text, which is then processed to carry out the desired actions within the email client. This system is designed to work in real-time, offering a seamless experience where users can simply speak to control their email accounts.

To enhance user interaction, the system also incorporates text-to-speech (TTS) functionality, providing auditory feedback to users. When an email is read, for instance, the system vocalizes the content, ensuring that blind users can access their inbox and understand email content. The architecture relies on commonly available speech recognition APIs, such as Google Speech-to-Text, and integrates them with email protocols for sending and receiving messages. The paper emphasizes the practical advantages of this voice-based system, particularly in empowering blind individuals to use technology independently. However, the authors acknowledge challenges such as ensuring high accuracy in speech recognition under diverse conditions (e.g., varying accents and background noise) and propose further improvements, including the need for robust error handling and the addition of more advanced natural language processing techniques for better contextual understanding.

[5] Danish, Kunwar, et al. "Speech recognition based E-mail for visually impaired." AIP Conference Proceedings. Vol. 3075. No. 1. AIP Publishing, 2024.

The paper "Speech Recognition Based E-mail for Visually Impaired" by Kunwar Danish and colleagues, published in the AIP Conference Proceedings in 2024, presents a speech-controlled email system specifically designed to assist visually impaired individuals in managing their emails. The system utilizes speech recognition technology to interpret voice commands, enabling users to compose, read, and send emails hands-free. It is intended to replace traditional keyboard and mouse-based interactions with voice commands, thus improving the accessibility of email services for people with visual disabilities. The authors aim to create a more intuitive and independent email management experience, reducing the reliance on screen readers or manual input methods, which can often be challenging or inefficient for visually impaired users.

In terms of technical implementation, the system integrates automatic speech recognition (ASR) to convert user speech into text and employs text-to-speech (TTS) technology to provide feedback, ensuring that the user is informed of system responses and email content. The system is built on open-source speech recognition frameworks, which provide flexibility and scalability for various user requirements. The authors also highlight the system's ability to handle basic email functions, such as composing new emails, reading incoming messages, and managing inbox content through simple voice commands. Despite its promising functionality, the study points out challenges related to enhancing speech recognition accuracy, particularly in noisy environments, and ensuring the system can handle a wide range of accents and speech patterns. The authors suggest further improvements in the natural language processing (NLP) capabilities to refine command recognition and make the system even more robust for diverse users.

2.3 Overview

The five works reviewed in the context of voice-controlled email systems for visually impaired users or individuals with disabilities explore innovative applications of speech recognition and text-to-speech technologies to enhance accessibility. A common theme across these studies is the effort to eliminate the barriers faced by visually impaired individuals when interacting with digital devices, especially email systems, by leveraging voice commands to control the process. In “Controlling email system using audio with speech recognition and text to speech” by Shah, Sundas, and Sharma (2021), the authors propose a system that integrates speech recognition with TTS, allowing users to interact with their email system entirely through voice. The system’s goal is to reduce dependency on visual interfaces, offering a more accessible solution for individuals who may struggle with traditional interaction methods. The paper outlines the potential benefits of integrating ASR and TTS technologies, which offer both hands-free operation and real-time auditory feedback, making the email system suitable for users with limited mobility or vision impairments.

In a similar vein, “Voice Controlled Email System Using Speech Recognition” by Umbare et al. focuses on enabling individuals to manage their email tasks, such as composing, sending, and reading messages, using spoken commands. The study emphasizes the practicality of the system in providing an accessible solution that simplifies email communication for users who are unable to rely on traditional input methods. The authors integrate speech recognition with Python-based scripting and APIs to perform email-related functions, but they also note challenges such as ensuring high recognition accuracy and the need for robust command processing to handle diverse linguistic input. Moreover, they highlight the potential for this system to be used in environments where multitasking or hands-free operations are crucial, but also emphasize the limitations in handling complex language constructs, background noise, and accents.

Moving towards AI-powered approaches, “AI Based Email System using Speech Recognition” by Umbare et al. introduces the integration of artificial intelligence with speech recognition to further enhance the system’s capabilities. By embedding AI into the email system, the researchers aim to make the system more adaptive to user needs, offering more sophisticated command recognition and improving contextual understanding. The paper also explores the potential for these AI-based systems to evolve over time, learning from interactions to enhance accuracy and personalization. However, the work acknowledges challenges similar to those in previous studies, including noisy environments and the need for robust natural language processing to deal with the complexities of human speech. The use of AI in this context allows for better adaptability and scalability of the system, paving the way for smarter, more efficient interactions with email services.

The work by Chimkode and Fatima, titled “Voice Based E-Mail System For Blind People Using Speech Recognition Technology”, presents a system that specifically targets the needs of blind individuals by using voice commands for controlling an email system. The system relies on speech-to-text (STT) and text-to-speech (TTS) technologies, which allow users to dictate commands and receive auditory feedback, providing them with an efficient means to interact with their email accounts. The system is designed to improve the independence of visually impaired users by offering a simple and intuitive interface that eliminates the need for traditional screen readers or manual input methods. While the work proves the concept of voice-based email management, the authors also acknowledge limitations related to speech recognition accuracy and voice command interpretation in diverse environments, suggesting further improvements in both user experience and system robustness.

CHAPTER 3

System Analysis

20-36

- 3.1. Introduction
- 3.2 Existing System & Disadvantages
- 3.3 Proposed System & Advantages
- 3.4 Software Requirements
- 3.5 Hardware Requirements
- 3.6 Modules Description
 - 3.6.1. Data Collection & Preprocessing
 - 3.6.2. Model Building
 - 3.6.3 Model Evaluation Metrics
- 3.7 System Architecture
- 3.8 Feasibility Study
 - 3.8.1 Economic Feasibility
 - 3.8.2. Technical Feasibility
 - 3.8.3. Procedural Feasibility
- 3.9. System Requirements
 - 3.9.1 Hardware Requirements
 - 3.9.2 Software Requirements

3. SYSTEM ANALYSIS

3.1 Introduction

System Analysis serves as a critical phase in the system development life cycle (SDLC), laying the foundation for successful design and implementation. It involves a comprehensive study of the current system be it manual, automated or semi-automated to identify its components, functionalities, limitations and areas for improvement. The primary objective of system analysis is to thoroughly understand the problem domain and gather detailed requirements that align with the goals of the intended system. Analysts collaborate closely with stakeholders including end-users, clients, developers and project managers to ensure that the new system will address existing gaps and enhance overall efficiency. This stage also involves feasibility studies, which assess whether the project is technically possible, economically viable, legally permissible and operationally acceptable.

In the context of modern software development, system analysis also involves modeling and documenting the functional and non-functional requirements of the system. Various tools and techniques, such as Data Flow Diagrams (DFDs), Entity-Relationship Diagrams (ERDs) and Use Case Diagrams, are employed to visualize system components and their interactions. This structured approach helps in decomposing complex systems into manageable modules, ensuring that every element is analyzed for potential risks and dependencies. Special attention is given to data input and output formats, user interaction workflows, performance criteria, security considerations, and scalability requirements. By doing so, analysts are able to outline a system blueprint that can be followed by designers and developers with minimal ambiguity. Moreover, system analysis bridges the gap between problem identification and solution architecture by providing a well-documented and validated understanding of user needs.

3.2 Existing System & Disadvantages

Currently, the majority of email platforms such as Gmail, Yahoo Mail, and Outlook are designed with a strong reliance on graphical user interfaces (GUI), making them inherently less accessible for visually impaired users. Although these systems can be accessed through assistive technologies like screen readers (e.g., JAWS, NVDA, or VoiceOver), the experience remains far from seamless. Users must remember a series of keyboard shortcuts, navigate through multiple layers of visual content, and manually manage tasks like email composition or inbox filtering, which can be overwhelming and cognitively demanding. Screen readers can only interpret what is displayed visually; they are reactive tools, not proactive interfaces. Additionally, they often do not offer contextual understanding or real-time correction, leading to a frustrating user experience, especially during tasks such as drafting professional emails. Furthermore, mainstream email services do not provide native speech-to-text input or voice navigation features beyond basic dictation on mobile platforms, which are frequently inaccurate and not optimized for visually impaired communication workflows.

Some standalone applications and plugins have attempted to offer limited voice-based email interactions, but these solutions typically lack integration with mainstream mail servers and often fail to provide a secure, reliable user experience. They do not offer full functionality for instance, users might be able to dictate an email but not read one back aloud using Text-to-Speech, or they may access the inbox but cannot filter, delete, or reply through voice commands. In most cases, these systems are not built with accessibility at the core of their architecture, resulting in patchy user journeys. Additionally, most do not support advanced language processing features, such as grammar correction or contextual suggestions, which are crucial for making spoken inputs professional and error-free. These fragmented approaches have left a major gap in truly inclusive and independent digital communication for blind and visually impaired individuals.

Disadvantages

- **Dependency on Visual Interfaces** Most email platforms rely heavily on graphical user interfaces, making it difficult for blind users to navigate without sighted assistance.
- **Limited Screen Reader Efficiency** Screen readers can be slow, complex, and often fail to convey the complete context or structure of an email, leading to confusion.
- **Lack of Voice Navigation** Mainstream email services do not support full voice-based navigation for all features like inbox access, email management, or settings.
- **No Built-In Speech Recognition** Users cannot naturally dictate emails or perform tasks through speech alone; third-party tools are needed, which may not be reliable or integrated.
- **No Real-Time Text Correction** When using basic speech-to-text tools, there's no built-in grammar correction or NLP support, resulting in unprofessional email content.
- **Manual Login Process** Password entry and multi-step login processes are difficult for visually impaired users, especially on graphical platforms.
- **Inaccessible Attachments Handling** Existing systems do not offer voice-based handling of attachments, making file uploading or downloading cumbersome for blind users.
- **Fragmented Solutions** Some tools support voice input, while others read text aloud, but very few offer an all-in-one solution combining both functionalities efficiently.

3.3 Proposed System &Advantages

The proposed system aims to bridge the accessibility gap in email communication for visually impaired users by providing a completely voice-controlled email platform. This system eliminates the need for visual interfaces by using speech recognition technology, allowing users to interact with the application entirely through voice commands. Users can create accounts, log in securely using voice authentication, and perform all essential email operations such as composing, sending, reading, replying to, and deleting emails all by speaking. The system uses Natural Language Processing (NLP) to accurately transcribe spoken input into grammatically correct and contextually appropriate text, ensuring professionalism and clarity in communication. Furthermore, with Text-to-Speech (TTS) integration, users can listen to received messages, making the process of checking and understanding emails seamless and intuitive. Unlike traditional screen readers, this system is proactive, user-friendly, and does not require the memorization of complex keyboard shortcuts or reliance on visual cues.

This solution also incorporates a custom mail server with a SQL database to handle user authentication, manage email data, and ensure secure storage and retrieval. The backend is optimized to respond to natural voice commands, allowing users to navigate their inbox, outbox, and other folders smoothly. Error handling and feedback mechanisms are provided via voice prompts, keeping users informed throughout the process. The logout process can also be triggered through voice, maintaining privacy without the need for manual steps. The entire platform is designed to be lightweight, accessible, and scalable, making it suitable for use on a variety of devices including desktops, laptops, and mobile phones. By focusing on autonomy, ease of use, and inclusivity, this voice-controlled email system empowers blind and visually impaired individuals to manage digital communication independently and confidently, promoting a more inclusive digital society.

Advantages

- **Complete Hands-Free Operation** Users can perform all email-related tasks using just their voice, eliminating the need for a keyboard or mouse and making it ideal for visually impaired users.
- **Enhanced Accessibility** The system is specifically designed for blind and visually impaired individuals, offering a user-friendly alternative to complex screen readers.
- **Accurate Voice-to-Text Transcription** With integration of Natural Language Processing (NLP), the system transcribes spoken words into grammatically correct, professional-quality email content.
- **Voice-Based Email Navigation** Users can navigate through inbox, sent mails, and trash folders using simple voice commands, ensuring an intuitive and smooth experience.
- **Text-to-Speech (TTS) Integration** Incoming emails can be read aloud, allowing users to listen to messages clearly without needing visual input.
- **Secure Voice Authentication** The system includes secure voice-based login and logout features, enhancing privacy and reducing reliance on typing passwords.
- **Efficient Error Handling** Voice feedback and confirmation prompts help users detect and correct errors easily, increasing system usability and reliability.
- **No Need for Visual Interface** The system removes the need for screen-based interaction, removing barriers posed by graphical user interfaces.

3.4 Software Requirements

- **Python (Version 3.11.4)** The primary programming language used for model development, data preprocessing, backend logic, and integration tasks.
- **TensorFlow / Keras** Deep learning libraries used to build, train, and evaluate the Convolutional Neural Network (CNN) for plant classification.
- **NumPy & Pandas** Essential Python libraries for handling numerical operations and data manipulation during preprocessing and analysis.
- **Matplotlib / Seaborn** Used for visualizing dataset distributions, training accuracy, loss curves, and other performance metrics.
- **OpenCV / PIL (Python Imaging Library)** Used for image loading, resizing, normalization, and basic image preprocessing tasks.
- **Scikit-learn** Utilized for preprocessing utilities (like label encoding, train-test splitting), evaluation metrics, and model validation techniques.
- **Streamlit** Web application framework used to create a user-friendly and interactive interface for real-time plant image uploads and results display.
- **Google Colab / Jupyter Notebook** Platforms used for initial model development, training, visualization, and experimentation in an interactive environment.

3.5 Hardware Requirements

- **Processor (CPU) Minimum:** Intel i5 or AMD Ryzen 5 Recommended for handling real-time computations, preprocessing tasks, and running the Streamlit interface smoothly.
- **Graphics Processing Unit (GPU)** Optional but recommended for training the CNN model efficiently. A GPU like NVIDIA GTX 1050 or higher can significantly reduce training time.
- **RAM (Memory) Minimum:** 8 GB Recommended: 16 GB or more, to support large dataset loading, model training, and running multiple background processes simultaneously.
- **Storage (Hard Drive / SSD) Minimum:** 256 GB HDD Recommended: 512 GB SSD, for faster data access, saving preprocessed images, model checkpoints, and system responsiveness.
- **Display Monitor** A standard display (HD or higher) is required for running and interacting with the Streamlit web interface or viewing outputs.
- **Internet Connectivity** Required for accessing YouTube videos, deploying the Streamlit app online, and retrieving updates or packages.
- **Power Supply / UPS** A stable power source is essential during model training or real-time predictions to avoid interruptions and potential data loss.
- **Peripheral Devices** Keyboard and mouse for interacting with the system during training, testing and application usage.

3.6 Modules Description

The proposed system is composed of several interconnected modules, each designed to carry out specific functions to deliver a seamless and accessible email experience for visually impaired users. The first and foremost module is the Speech Recognition Module, which is responsible for converting user voice input into text. This module leverages powerful APIs or models capable of recognizing natural language commands accurately, regardless of accents or minor speech variations. Users can create accounts, log in, compose emails, and execute various functions by speaking naturally. Once the voice is converted into text, the Natural Language Processing (NLP) Module comes into play. This component ensures that the transcribed text is grammatically correct, contextually meaningful, and professionally structured. It handles common issues like mispronunciations or filler words and improves the final email content's clarity and tone.

Next, the Text-to-Speech (TTS) Module is used to read out emails from the inbox or notifications back to the user. This module helps users consume content without needing to view the screen, offering a responsive and natural-sounding voice interface. The Mail Management Module handles backend processes like composing, sending, replying to, deleting, and retrieving emails. It interacts with a custom-built SQL database that stores user credentials, emails, and related metadata securely. This module also includes session management and voice-based logout to ensure user privacy. The Authentication Module supports secure login via voice recognition or voice commands, eliminating the need for typing credentials. Finally, the Error Handling and Feedback Module ensures that the system responds with appropriate voice prompts when a command is unclear, invalid, or when an operation is successfully completed. Together, these modules function harmoniously to provide a fully voice-driven, inclusive, and user-friendly email system tailored for the visually impaired.

3.6.1 Data Collection Process

The data collection phase is essential for training and fine-tuning both the speech recognition and natural language processing components of the proposed system. For the speech-to-text functionality, diverse voice datasets are gathered, which include recordings of people giving email-related commands in various accents, speech speeds, and environments. Open-source datasets such as Mozilla's Common Voice or Google's Speech Commands dataset can be leveraged, along with custom voice inputs recorded to simulate real-world usage. These recordings help the model understand variations in human speech patterns, enabling accurate transcription. Similarly, email content samples are collected to train the NLP module—these include professional and casual email texts, allowing the system to learn grammar correction, context understanding, and sentence restructuring. The collected text data is labeled and categorized to enhance the system's ability to convert unstructured spoken input into well-structured, formal written content.

The preprocessing stage involves preparing both the audio and text data for efficient model training and real-time implementation. Audio files are cleaned to remove background noise, normalized to maintain volume consistency, and segmented into manageable lengths for accurate parsing. Techniques like silence removal, Mel-frequency cepstral coefficients (MFCC), and feature extraction are applied to make the speech data machine-readable. On the other hand, text data undergoes preprocessing steps such as tokenization, lemmatization, punctuation removal, and stop word filtering. For NLP correction, pre-trained models are fine-tuned using this clean, processed data to ensure that grammar correction and contextual understanding are optimized for real-time dictation scenarios. The preprocessing also includes filtering out ambiguous or irrelevant data and ensuring that inputs to the models are consistent and standardized. This careful preparation ensures high accuracy, fast response times, and a smooth, natural interaction experience for the end user, making the system robust, adaptive, and reliable for blind and visually impaired users.

3.6.2 Module Techniques

The core operations of the proposed voice-controlled email system revolve around enabling users to interact with email services entirely through vocal commands. Once the system is launched, the user is prompted with a voice-based interface that asks whether they want to log in, sign up, or exit. If a new user chooses to register, the system collects credentials like name and email ID using voice prompts, and stores this information securely in a SQL database. Returning users can log in by speaking their credentials, which are verified using voice or password-based authentication. Upon successful login, users can issue commands like “Compose Mail,” “Read Inbox,” or “Delete Email.” These commands are passed to the Speech Recognition Module, which processes them and routes them to the appropriate function in the system. When composing an email, users speak out the recipient’s name or address, subject, and message. The NLP module corrects grammar and formats the email properly before sending. The backend Mail Handling Module ensures that emails are sent or saved in the appropriate folders, just like a traditional system but without visual dependency.

In addition to sending emails, the system also supports operations like reading received messages aloud through a Text-to-Speech (TTS) engine. When the user asks to check their inbox, the system retrieves the emails from the database and converts the content into speech, reading each email out loud clearly. Users can navigate between emails using commands like “Next Email,” “Previous Email,” or “Repeat.” They can reply to any message using the “Reply” command, after which the voice input is transcribed and structured into a reply format. Deleting, forwarding, or saving drafts are also done via simple spoken instructions. If a user makes a mistake or the system misinterprets a command, it provides audio feedback and options for correction or reattempt. The session continues until the user says “Logout,” which safely ends the session while maintaining data integrity. These operations are designed to be smooth, intuitive, and efficient, allowing visually impaired users to handle all essential email tasks independently without relying on any graphical user interface.

3.6.3 Model Evaluation Metrics

Evaluating the performance of the underlying models in the voice-controlled email system is crucial to ensure reliable, accurate, and user-friendly functionality. For the Speech Recognition Module, key evaluation metrics include Word Error Rate (WER), Sentence Error Rate (SER), and Real-Time Factor (RTF). Word Error Rate is a standard measure in speech-to-text tasks, calculated based on the number of insertions, deletions, and substitutions needed to transform the predicted transcription into the actual text. A lower WER indicates a more accurate model, which is especially important for users relying solely on voice input. Sentence Error Rate evaluates the percentage of complete sentences that are incorrectly transcribed, which is vital in understanding how well the system processes complex voice commands or entire email content. Additionally, the Real-Time Factor measures how quickly the model can process speech input compared to the time it takes to speak it, directly affecting user experience and responsiveness. A low RTF is preferred to maintain smooth interactions without lags.

For the Natural Language Processing (NLP) Module responsible for grammar correction and context-aware email generation, metrics such as BLEU Score, ROUGE Score, Grammatical Error Correction Rate, and Semantic Similarity are used. The BLEU (Bilingual Evaluation Understudy) score quantifies how closely the machine-generated corrections resemble human-written sentences, making it ideal for ensuring high-quality email outputs. ROUGE (Recall-Oriented Understudy for Gisting Evaluation) score helps assess the overlap of content before and after correction, indicating how much original meaning is retained. Grammatical Error Correction Rate focuses on identifying how many grammar issues the system can correct in a given input, measuring the practical usefulness of the NLP model. Finally, semantic similarity metrics ensure that even after corrections, the intended message or context of the email is preserved. These evaluation metrics collectively help maintain the integrity, fluency, and efficiency of the email creation and communication process, ensuring the solution is not only functional but also accurate and dependable for the visually impaired.

3.7 System Architecture

The system architecture of the voice-controlled email platform for visually impaired users is designed as a modular, layered structure that ensures seamless interaction between user voice input and backend email functionalities. At the core lies the Voice Interaction Layer, where users communicate through vocal commands, captured and processed using Speech Recognition APIs that convert audio into text. This is followed by the Command Processing and NLP Layer, which interprets user intent, performs grammar corrections, and formulates structured email content using natural language processing models. These inputs are passed to the Application Logic Layer, which includes modules for email composition, inbox management, replying, deletion, and account authentication. All email-related actions are coordinated through a Custom Mail Server linked to a SQL Database, which securely stores user credentials, messages, and transaction history.

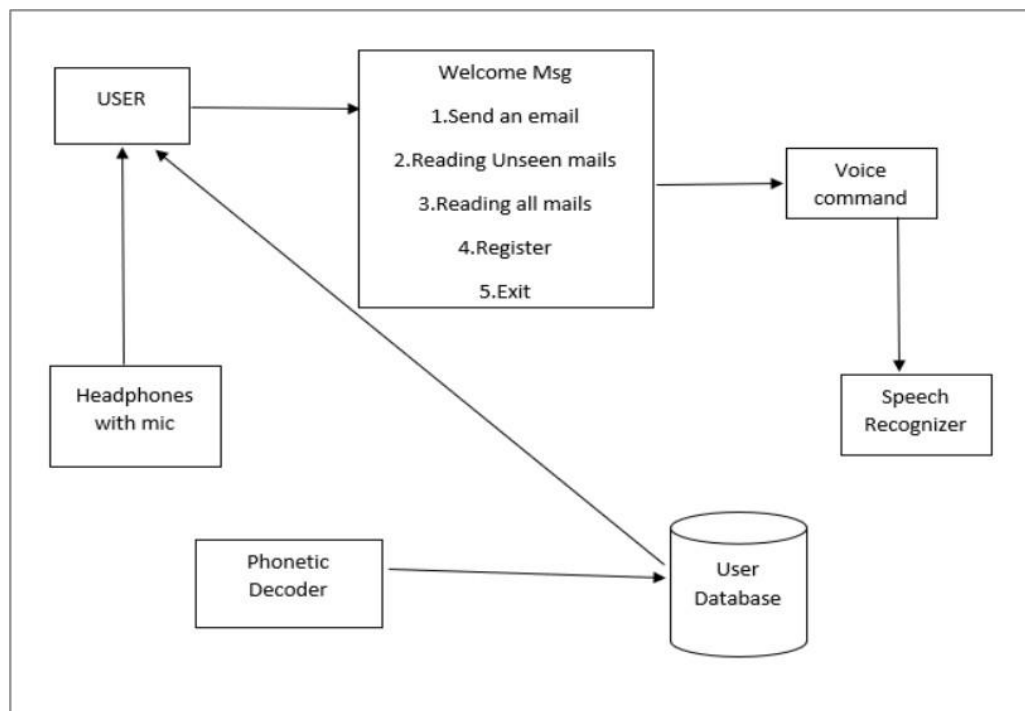


Fig.1 System Architecture

3.8 Feasibility Study

A Feasibility Study is a critical step in the early stages of system development, conducted to assess the practicality, viability, and overall potential for success of a proposed project. It acts as a risk assessment tool that evaluates whether the project should proceed, be modified, or be abandoned altogether. The feasibility study involves a multidimensional analysis covering technical, operational, economic, legal, and scheduling factors. By systematically evaluating each aspect, stakeholders can make informed decisions about the investment of time, money and resources. The primary goal of the feasibility study is to ensure that the project is achievable within the given constraints and that it can deliver the intended benefits without significant obstacles or risks. Technical feasibility focuses on whether the required technology, tools, and skills are available to successfully develop and deploy the system.

This includes evaluating the technical expertise of the development team, compatibility with existing systems, availability of platforms (hardware/software), and scalability of the solution. If the existing technical environment cannot support the proposed system, necessary upgrades or alternatives are explored. Operational feasibility examines how well the proposed solution aligns with the organizational processes and user needs. It considers whether the system will be accepted and adopted by users, and whether it fits within the current workflow. The goal is to determine if the organization is ready to adapt to the new system and if training, maintenance, and support structures can be effectively implemented. Economic feasibility, also known as cost-benefit analysis, evaluates whether the project is financially worthwhile. This involves estimating development costs, operational costs, maintenance expenses, and comparing them to the expected financial or qualitative benefits, such as increased productivity, efficiency or user satisfaction. A project is economically feasible if the projected benefits outweigh the associated costs within a reasonable time frame.

3.8.1 Economic Feasibility

Economic Feasibility is a crucial component of the feasibility study that assesses whether the proposed project is financially viable and justifiable in terms of cost and benefit. It involves a detailed cost-benefit analysis to estimate all the potential expenses associated with the system's development, deployment, and maintenance, including hardware and software acquisition, labor, infrastructure, training, and ongoing operational costs. On the benefits side, it considers both tangible and intangible gains such as increased productivity, reduced operational time, improved data accuracy, enhanced user satisfaction, better decision-making, and potential revenue generation. The analysis compares the total costs against the expected returns over a defined period to determine the break-even point and return on investment (ROI). A project is considered economically feasible if it promises significant benefits that outweigh the costs and if it aligns with the budgetary constraints and financial strategy of the organization.

3.8.2 Technical Feasibility

Technical Feasibility evaluates whether the current technology, infrastructure, and technical expertise are sufficient to support the development and implementation of the proposed system. It involves assessing the availability and compatibility of the required hardware, software, programming languages, tools, frameworks and platforms necessary to build and maintain the system efficiently. This includes determining whether the existing technical environment can integrate with or support the new solution without requiring major modifications or upgrades. The analysis also considers the technical skills and experience of the development team, the complexity of the system architecture, the reliability and scalability of the chosen technologies, and any potential technical challenges or limitations. A project is deemed technically feasible if the organization possesses or can acquire the necessary resources and competencies within a reasonable time and cost.

3.8.3 Procedural Feasibility

Procedural Feasibility refers to the evaluation of how well the proposed system aligns with the existing policies, procedures, and workflows within an organization. It examines whether the new system can be integrated smoothly into the current operational framework without causing major disruptions or conflicts. The goal is to ensure that the system supports organizational objectives while conforming to established rules, business logic, and administrative processes. If the proposed solution requires changes in procedures this feasibility study also analyzes whether such changes are practical, acceptable to stakeholders and beneficial in the long term. Understanding the procedural alignment helps minimize resistance to change and facilitates smoother adoption of the system across various departments. An important aspect of procedural feasibility is assessing user readiness and organizational culture. It considers whether employees and stakeholders are prepared and willing to adapt to new workflows or altered responsibilities that may result from system implementation.

This includes evaluating the need for training programs, procedural documentation and support systems to ensure smooth transition and ongoing usability. Resistance to change can be a major hurdle in many organizations, especially when established workflows are deeply embedded in the daily operations. Hence, procedural feasibility also involves conducting stakeholder interviews, process mapping, and risk assessment to identify potential bottlenecks or opposition that could impede successful deployment. It also checks whether the new procedures adhere to legal and ethical guidelines specific to the domain of the application. Furthermore, procedural feasibility involves establishing clear governance and operational control mechanisms for the new system. It defines the flow of tasks, roles, responsibilities and escalation points within the updated process landscape. This helps in creating a structured approach for monitoring system performance and ensuring accountability post-deployment.

3.9 System Requirements

3.9.1 Hardware Requirements

1. **Processor** – Intel Core i5 or above
2. **RAM** – Minimum 8GB
3. **GPU** – NVIDIA GTX/RTX series (optional but recommended)
4. **Storage** – Minimum 256GB SSD
5. **Monitor** – HD Display (1920x1080 resolution)
6. **Keyboard** – Standard QWERTY Keyboard
7. **Mouse** – Optical Mouse
8. **Internet Connectivity** – Stable broadband connection
9. **Power Supply** – Reliable power source with backup (UPS)

3.9.2 Software Requirements

1. **Operating System** – Windows 10/11, Linux (Ubuntu), or macOS
2. **Programming Language** – Python 3.11 or above
3. **Deep Learning Library** – TensorFlow / Keras
4. **Image Processing Library** – OpenCV
5. **Data Handling Libraries** – NumPy, Pandas
6. **Visualization Tools** – Matplotlib, Seaborn
7. **Evaluation Tools** – Scikit-learn
8. **Browser** – Google Chrome / Firefox / Edge
9. **IDE** – VS Code

CHAPTER 4

System Design

37-45

4.1. Introduction

4.2. DFD Diagrams

4.3 UML Diagrams

4.3.1 Use Case Diagram

4.3.2 Sequence Diagram

4.3.3 Class Diagram

4.3.4 Collaboration Diagram

4.3.5 Activity Diagram

4. SYSTEM DESIGN

4.1 Introduction

System Design is a fundamental phase in the software development life cycle that translates the requirements gathered during the analysis phase into a blueprint for building the actual system. It involves the conceptualization and structuring of system components, including data architecture, software modules, user interfaces, and system workflows. The primary objective of system design is to create a comprehensive plan that guides developers, ensures clarity in development tasks, and aligns with both functional and non-functional requirements. This phase acts as a bridge between what the system is supposed to do (as identified during analysis) and how those functions will be technically implemented. Effective system design not only supports the achievement of system goals but also enhances maintainability, scalability, performance and user experience.

It includes high-level design (HLD), which outlines the system architecture, data flow, and module relationships and low-level design (LLD) which details specific implementation logic, algorithms, database schema and interface layouts. Moreover, system design is vital for identifying and managing interdependencies among system components, minimizing redundancy, and ensuring efficient data processing and communication across modules. It considers critical aspects such as system security, modularity, reusability, fault tolerance and future extensibility. By leveraging design principles and methodologies like object-oriented design, component-based architecture, and model-driven development, designers ensure the system meets technical and user requirements within the constraints of cost and time. System design also defines the communication mechanisms between internal modules and external systems, ensures database integrity through normalization and incorporates user-centric features that make the system intuitive and accessible.

4.2 DFD Diagrams

Data Flow Diagrams (DFDs) are essential tools used in the system design phase to visually represent how data moves through a system, how it is processed, and how it interacts with external entities and internal processes. They provide a clear and structured way to understand the flow of information, showcasing inputs, outputs, storage points and processing stages within the system. DFDs are typically organized in levels starting from the Context Level (Level 0) which gives a high-level overview of the entire system as a single process interacting with external entities followed by Level 1 and Level 2 diagrams which progressively break down the main system into detailed subprocesses and data stores. These diagrams help identify inefficiencies, bottlenecks, and redundancies in data movement, making them invaluable for both developers and stakeholders during the planning and refinement of system architecture.

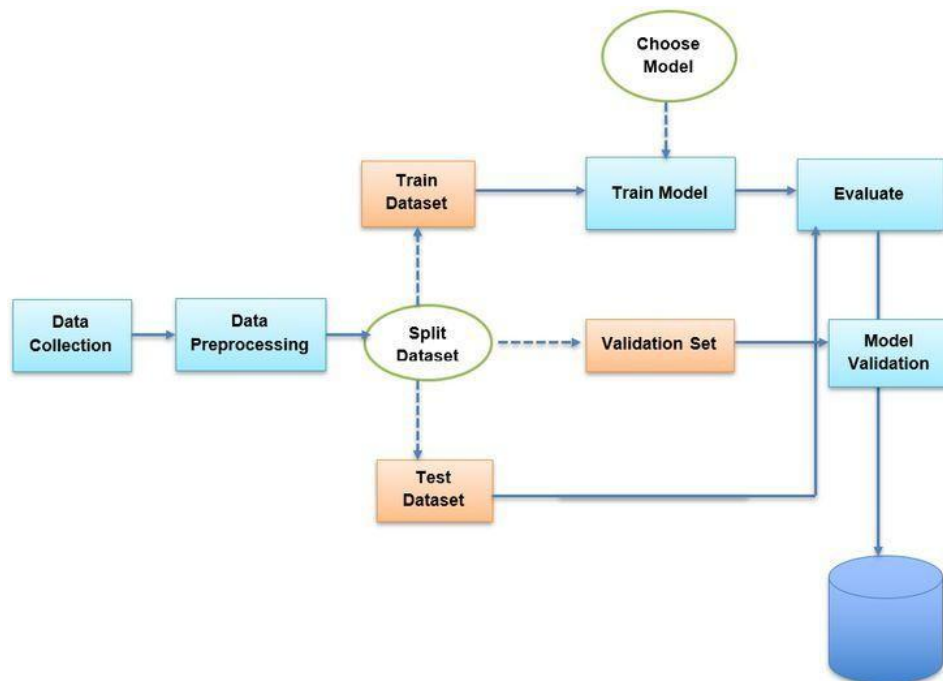


Fig.2 Data Flow Diagram

4.3 UML Diagrams

Unified Modeling Language (UML) Diagrams are powerful visual representations used in software engineering to model, design, and document the architecture and behavior of a system. They provide a standardized way to visualize the structure and dynamics of a system, making complex software systems easier to understand and communicate. UML diagrams are divided into two main categories: structural diagrams and behavioral diagrams. Structural diagrams, such as Class Diagrams, Object Diagrams, Component Diagrams, and Deployment Diagrams, focus on the static aspects of the system defining the system's objects, classes, relationships, and physical architecture. For instance, Class Diagrams describe the system's classes, attributes, methods, and the relationships among objects, helping developers understand how the codebase should be organized. These diagrams ensure that the design is logically sound, reusable, and aligns with object-oriented principles, facilitating clearer communication between team members during development.

On the other hand, behavioral UML diagrams focus on the dynamic aspects of the system how it behaves during operation. These include Use Case Diagrams, Sequence Diagrams, Activity Diagrams, and State Diagrams. Use Case Diagrams illustrate the interactions between users (actors) and the system, helping identify system functionalities and user goals. Sequence Diagrams map out the sequence of messages exchanged between objects in a particular interaction, capturing the order of operations and flow of logic. Activity Diagrams highlight workflows and decision-making processes within the system showing how control flows from one activity to another. State Diagrams track changes in the state of an object in response to events. Together, these diagrams offer a comprehensive view of system functionality and interaction, making it easier to plan, design, test, and maintain complex systems. UML diagrams thus serve as a blueprint for the development process, bridging the gap between conceptual design and actual implementation.

4.3.1 Use Case Diagram

Use Case Diagrams are a vital part of UML that visually depict the functional requirements of a system from the user's perspective. They illustrate the interactions between external entities, known as actors (such as users or other systems), and the system itself, represented through use cases, which are specific functions or services the system provides. Each use case encapsulates a distinct functionality or goal that an actor wants to achieve, such as "Login," "Search Products," or "Generate Report." These diagrams help identify the boundaries of the system, the roles of different users, and the major functionalities it must support. By focusing on what the system should do rather than how it does it, Use Case Diagrams are extremely helpful during the requirements-gathering phase and serve as a foundational reference for designing, testing, and validating the system.

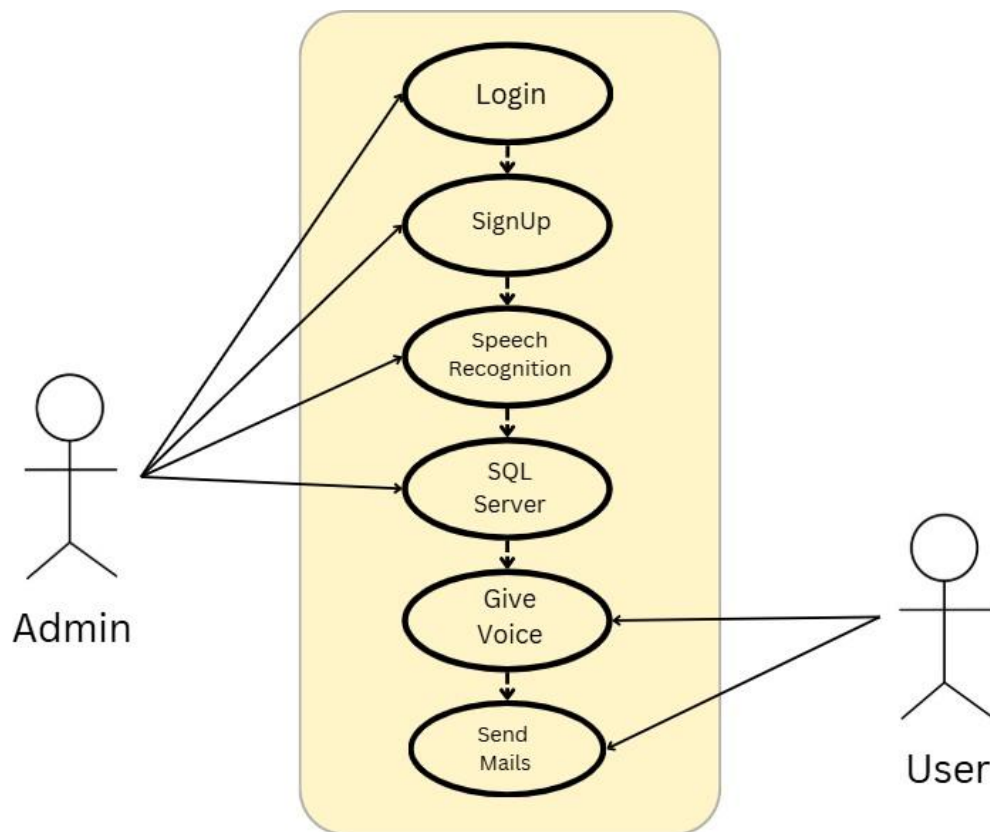


Fig.3 Use Case Diagram

4.3.2 Sequence Diagram

Sequence Diagrams are a type of UML behavioral diagram that model the interaction between different system components over time, focusing on the chronological order of message exchanges between objects or entities. They visually represent how and in what order the objects in a system interact to carry out a particular functionality or process. Each object is shown as a vertical lifeline, and horizontal arrows depict the messages passed between them, indicating the flow of control and data. Sequence diagrams are especially useful for understanding complex use cases, identifying potential issues in system logic and designing efficient communication between components. They help developers visualize the internal workings of a feature, clarify object responsibilities and ensure that the system behaves as intended in different scenarios.

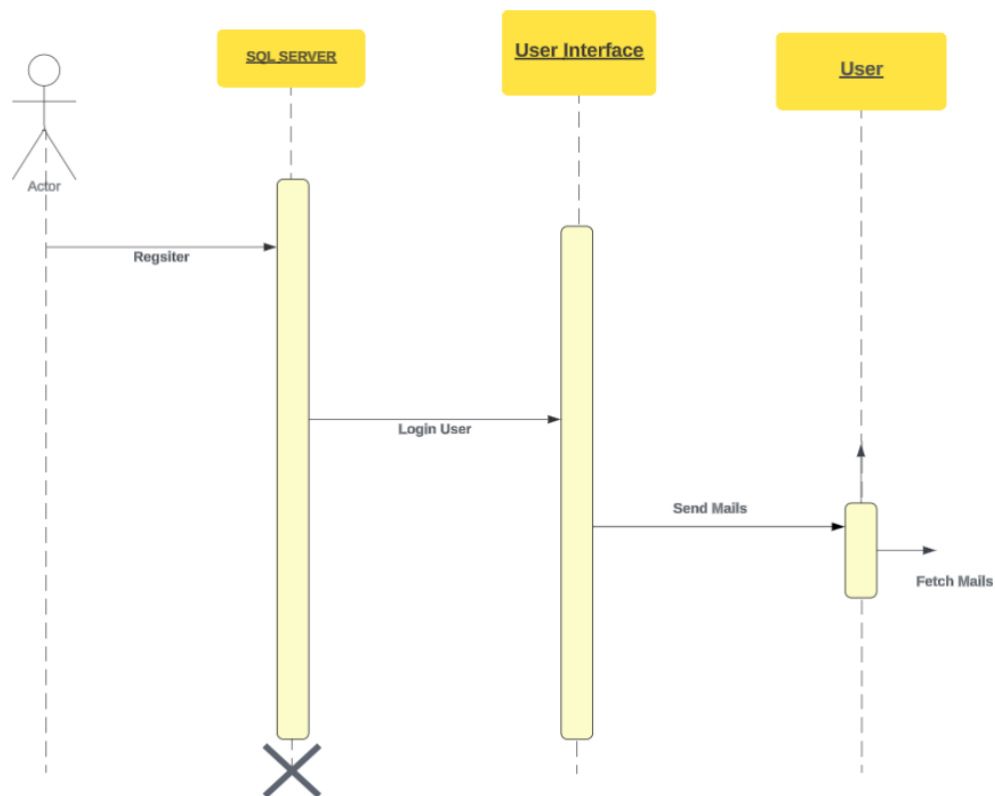


Fig.4 Sequence Diagram

4.3.3 Class Diagram

Class Diagrams are a fundamental component of UML used to represent the static structure of a system by modeling its classes, attributes, methods, and the relationships between them. Each class is depicted as a rectangle divided into three sections: the top for the class name, the middle for its attributes (or properties), and the bottom for its methods (or operations). These diagrams help visualize the system's object-oriented design by showing how different classes interact, inherit from each other (generalization), depend on each other (associations), or are composed of other classes (composition/aggregation). Class diagrams provide a clear blueprint for the software's architecture, supporting the implementation process by guiding developers on how to structure code and manage object interactions.

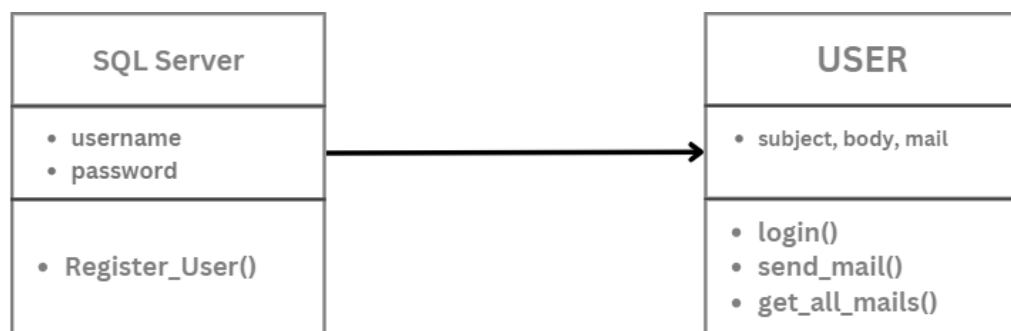


Fig.5 Class Diagram

4.3.4 Collaboration Diagram

Collaboration Diagrams, also known as Communication Diagrams, are a type of UML interaction diagram that illustrate how objects interact and collaborate to perform a specific functionality, emphasizing the structural organization of objects and the messages exchanged between them. Unlike Sequence Diagrams that focus on the time sequence of messages, Collaboration Diagrams highlight the relationships and links among the participating objects. Each object is represented as a rectangle, and messages are shown as labeled arrows with sequence numbers, indicating the flow and order of communication. These diagrams are especially useful for understanding object roles, responsibilities, and the overall interaction pattern in a system. By combining both the structural and behavioral aspects of interactions, Collaboration Diagrams provide a comprehensive view of how components work together, aiding in both system design and debugging by clarifying object coordination within various use cases.

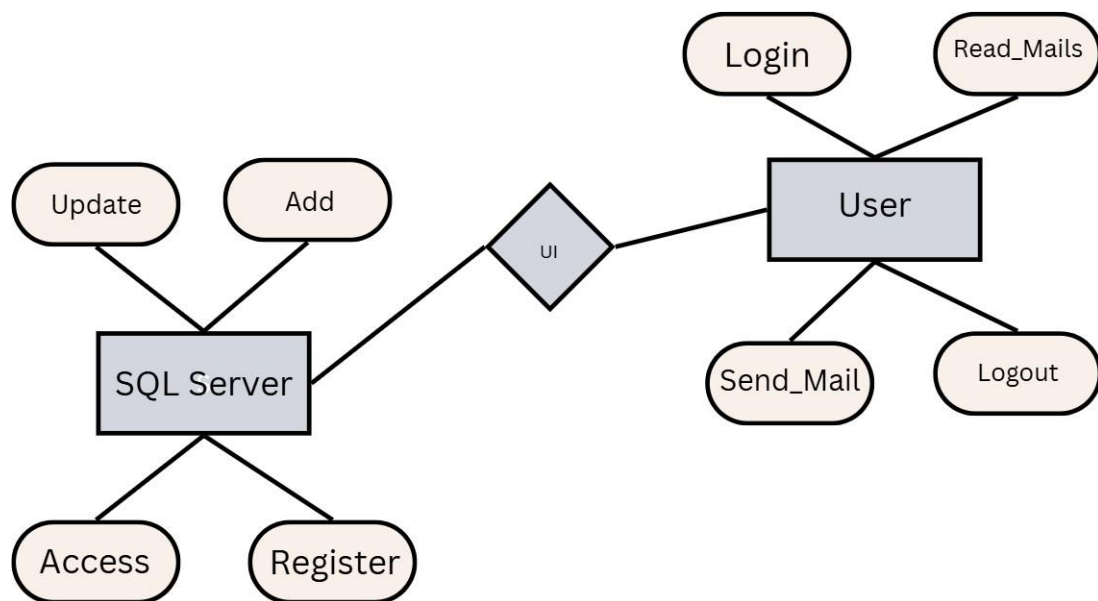


Fig.6 Collaboration Diagram

4.3.5 Activity Diagram

Activity Diagrams are a type of UML behavioral diagram that visually represent the flow of activities or actions within a system, showcasing the dynamic aspects of the system's behavior. They are particularly useful for modeling workflows, business processes or the logic behind a use case by illustrating the sequence of activities, decision points, parallel processes and the start and end states. Each activity is represented as a rounded rectangle, while control flows are shown as arrows connecting the steps, and decision nodes are depicted as diamonds. Activity diagrams help in understanding complex operations, identifying bottlenecks, and improving process efficiency by providing a clear, high-level overview of how tasks progress from one stage to another.

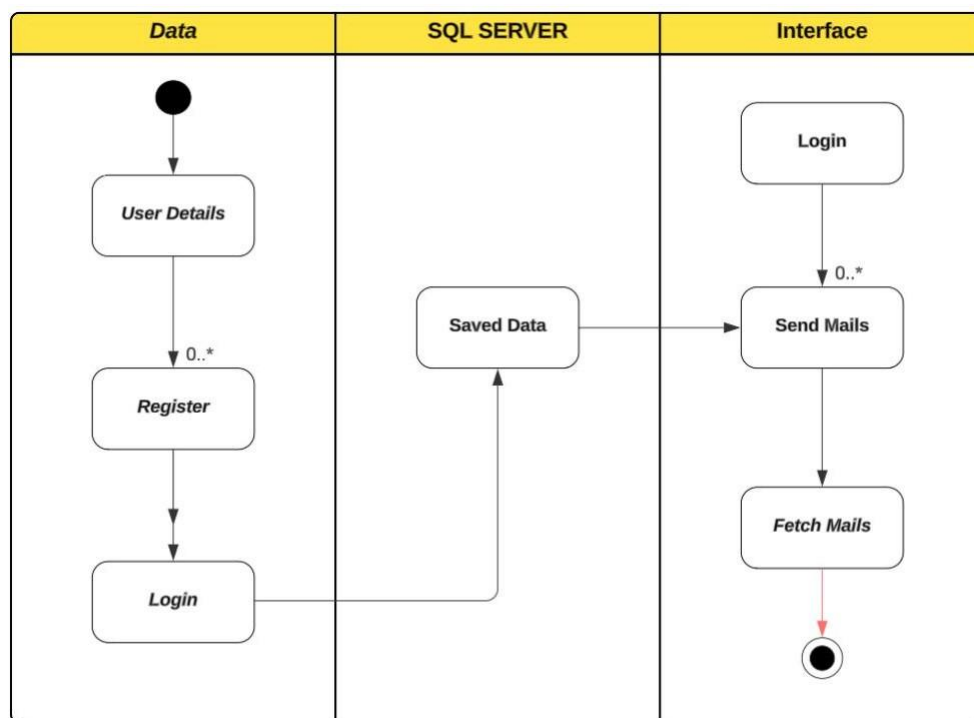


Fig.7 Activity Diagram

CHAPTER 5

System Coding & Testing

46-54

5.1. Introduction

5.2. Implementation

5.3 About Python

5.4 Sample Code

5.5 Software Testing Techniques

5.6 Software Testing Strategies

5. SYSTEM CODING & TESTING

5.1 Introduction

System Coding is the phase where the actual implementation of the system begins based on the design specifications laid out in earlier stages. This involves translating the system's design, including UML diagrams and data flow models, into a working software application using appropriate programming languages and frameworks. During this stage, developers write the source code for each module or component of the system, ensuring that it adheres to coding standards, is modular, and follows best practices for maintainability and scalability. Coding is typically done in an integrated development environment (IDE) where the system's functionalities are built incrementally, starting from core logic to UI components and database interactions. Proper documentation, version control, and commenting are maintained to support future enhancements and team collaboration. Developers also integrate third-party APIs, security layers and optimization techniques as required by the system to ensure high performance and robustness.

System Testing, on the other hand is a critical stage that follows coding and is aimed at validating and verifying that the developed software functions correctly, meets all specified requirements, and is free of bugs. It involves multiple levels of testing such as unit testing (testing individual components), integration testing (testing interactions between modules), system testing (testing the complete system for overall behavior), and acceptance testing (validating the system against end-user requirements). Test cases are prepared based on both functional and non-functional requirements, and each test is executed systematically to identify any discrepancies, logical errors, or performance issues. Automated and manual testing methods may be employed depending on the project scope. The feedback from testing is used to debug and refine the system, ensuring that it is reliable, secure, and ready for deployment.

5.2 Implementation

The implementation of the voice-controlled email system begins with setting up the backend infrastructure that supports user authentication, email management, and voice processing. The development process starts with creating a custom user registration and login system using Python and a SQL database to store credentials and user-specific email data securely. The user interface is built using Flask or Streamlit, offering voice-only navigation to remove the need for visual interaction. Once logged in, users are greeted with a series of voice prompts that guide them through the functionalities such as composing, reading, replying, or deleting emails. The voice commands are captured using the SpeechRecognition library, which processes and converts the spoken input into textual data. This data is parsed to understand the user's intent, allowing for context-aware operations. During email composition, the dictated message is passed through Natural Language Processing (NLP) libraries like TextBlob or spaCy for grammar and spelling correction, ensuring that the final email content is both readable and accurate before being dispatched.

In the next phase, the system utilizes a Text-to-Speech (TTS) engine, such as gTTS or pyttsx3, to read aloud incoming emails, system messages, and instructions to the user. This functionality is particularly crucial for users navigating their inbox or reviewing sent items. The interaction is designed to be continuous and conversational, allowing users to issue commands such as "Next email," "Reply," "Delete," or "Go back" without needing to restart the session. Each command triggers backend processes that interact with the SQL database to retrieve, update, or delete relevant email records. Voice-based session management is also implemented, enabling secure logouts and privacy control. Error handling mechanisms ensure that if a command is misunderstood, the system requests clarification and gracefully recovers from potential disruptions. The overall implementation integrates multiple modules into a cohesive, accessible platform that empowers visually impaired users to independently manage their email communication through speech alone, offering a truly inclusive digital experience.

5.3 About Python

Python is a high-level, interpreted and general-purpose programming language that emphasizes code readability and simplicity making it one of the most popular languages in the world today. Created by Guido van Rossum and first released in 1991, Python has gained immense popularity due to its clean syntax and ease of learning, which make it ideal for both beginners and experienced developers. Its dynamic typing and automatic memory management allow developers to write fewer lines of code compared to many other programming languages, without compromising functionality. Python supports multiple programming paradigms, including procedural, object-oriented, and functional programming, making it versatile for a wide range of applications such as web development, automation, software development and scientific computing.

One of Python's strongest assets is its vast and active community, which contributes to a rich ecosystem of libraries and frameworks. These include powerful libraries like NumPy, Pandas, Matplotlib, TensorFlow, Scikit-learn, Flask and Django, which enable developers to efficiently build solutions in data science, machine learning, artificial intelligence, web development, and more. Python's platform independence means code can run on any operating system with minimal or no modification. Additionally, Python integrates well with other languages such as C/C++ and Java, and supports extensive tools for debugging, testing, and packaging. Its scalability, flexibility, and support for rapid prototyping make Python a go-to language for startups, enterprises, and researchers worldwide.

5.4 Sample Code

```
import sqlite3

# Initialize the database

conn = sqlite3.connect("email_system.db")

cursor = conn.cursor()


# Create users table

cursor.execute("""CREATE TABLE IF NOT EXISTS users (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    email TEXT UNIQUE NOT NULL,

    password TEXT NOT NULL

)""")


# Create emails table

cursor.execute("""CREATE TABLE IF NOT EXISTS emails (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    sender TEXT NOT NULL,

    receiver TEXT NOT NULL,

    subject TEXT,

    body TEXT,

    status TEXT DEFAULT 'unread'

)""")

conn.commit()

conn.close()
```

```

# Functions to handle database operations

def add_user(email, password):

    conn = sqlite3.connect("email_system.db")

    cursor = conn.cursor()

    try:

        cursor.execute("INSERT INTO users (email, password) VALUES (?, ?)", (email,
password))

        conn.commit()

        return True

    except sqlite3.IntegrityError:

        return False

    finally:

        conn.close()


def verify_user(email, password):

    conn = sqlite3.connect("email_system.db")

    cursor = conn.cursor()

    cursor.execute("SELECT * FROM users WHERE email=? AND password=?",
(email, password))

    user = cursor.fetchone()

    conn.close()

    return user is not None


def store_email(sender, receiver, subject, body):

    conn = sqlite3.connect("email_system.db")

    cursor = conn.cursor()

```

```

        cursor.execute("INSERT INTO emails (sender, receiver, subject, body) VALUES
        (?, ?, ?, ?)",

                        (sender, receiver, subject, body))

    conn.commit()

    conn.close()

def fetch_unread_emails(user_email):

    conn = sqlite3.connect("email_system.db")

    cursor = conn.cursor()

    cursor.execute("SELECT * FROM emails WHERE receiver=? AND
status='unread'", (user_email,))

    emails = cursor.fetchall()

    conn.close()

    return emails

def mark_email_as_read(email_id):

    conn = sqlite3.connect("email_system.db")

    cursor = conn.cursor()

    cursor.execute("UPDATE emails SET status='read' WHERE id=?", (email_id,))

    conn.commit()

    conn.close()

def fetch_user(email):

    conn = sqlite3.connect("email_system.db")

    cursor = conn.cursor()

    cursor.execute("SELECT * FROM users WHERE email=?", (email,))

```

5.5 Software Testing Techniques

Unit Testing: Unit testing is a foundational testing technique where individual components or functions of a software application are tested in isolation. The main objective is to verify that each unit of the software performs as expected. Typically carried out by developers during the development phase, unit tests help detect bugs early in the development lifecycle, reducing the cost and effort of fixing issues later.

Integration Testing: After individual units are verified, integration testing is conducted to ensure that different modules or components of the software interact correctly when combined. This technique uncovers issues related to data flow between modules, incorrect interfaces, or incompatible logic. There are several approaches to integration testing, such as top-down, bottom-up, and big bang. This phase is crucial because even if units work perfectly in isolation, they may fail when integrated due to incorrect assumptions or interface mismatches.

System Testing: System testing is performed on the entire, integrated software system to validate its compliance with the specified requirements. This is a high-level test that simulates real-world scenarios to verify both the functional and non-functional aspects of the application. System testing includes various subtypes like usability testing, performance testing, and security testing. It is often conducted by a separate QA team rather than developers to ensure an unbiased evaluation.

Black Box Testing: Black box testing is a method where the tester evaluates the software without any knowledge of its internal code or structure. The focus is solely on input and output; the tester provides inputs and observes the outputs to verify correctness. This technique is widely used for functional testing and is beneficial for validating user interfaces, APIs, and system behavior against business requirements. Black box testing ensures that the application performs its intended functions and meets user needs without being influenced by how the internal logic is implemented.

5.6 Software Testing Strategies

Software Testing Strategies are systematic approaches used to plan, design, and execute testing activities to ensure that software meets its functional and non-functional requirements. These strategies define the scope, objectives, tools, environments, and techniques to be used throughout the testing process. Some commonly used strategies include Top-Down Testing, where high-level modules are tested first; Bottom-Up Testing, which starts with the lower-level modules; Big Bang Testing, where all components are integrated and tested together; and Incremental Testing, where components are integrated and tested step-by-step. Another widely adopted strategy is Agile Testing, which involves continuous testing throughout the development cycle, promoting early detection of bugs and faster feedback. These strategies help improve software quality, ensure complete test coverage, reduce risks, and align testing with project goals.

Test Case ID	Test Scenario	Actual Result	Expected One	Status
TC001	User gives command to log in with correct credentials	User logged in successfully	User logged in successfully	Pass
TC002	User gives incorrect login credentials via voice	System rejects login and prompts retry	System rejects login and prompts retry	Pass
TC003	User dictates an email to compose and send	Email transcribed, corrected, and sent	Email transcribed, corrected, and sent	Pass

Table.1 Test Cases

CHAPTER 6

RESULTS

55-58

6.1. Introduction

6.2. Output Screens

6. RESULTS

6.1 Introduction

The primary goal of this project was to design and implement a speech-based email communication platform that caters specifically to the needs of blind and visually impaired individuals. Traditional email systems heavily rely on visual interaction and keyboard/mouse input, which poses significant challenges for visually impaired users who often depend on screen readers or assistance from others. However, such approaches often fall short of providing a fully independent and hands-free user experience. With this project, a voice-controlled email environment was built that enables users to perform all essential email operations including login, email composition, inbox management, reading messages, and logout using just their voice. This system not only enhances independence but also ensures ease of use and accessibility through technologies like speech recognition, natural language processing (NLP), and text-to-speech (TTS).

The system has been carefully evaluated to validate its performance, accuracy, and usability in real-time environments. Users were able to successfully interact with the email system using intuitive voice commands, which were accurately transcribed and processed. Grammar correction through NLP ensured the composed messages were coherent and professional, while the TTS feature effectively read emails aloud to users with clear pronunciation. The results demonstrate the robustness and practicality of the proposed solution in bridging the accessibility gap in digital communication. By eliminating the need for graphical interfaces and physical input devices, this platform marks a significant step forward in inclusive technology and reinforces the importance of designing systems that accommodate the unique challenges faced by visually impaired users.

6.2 Output Screens

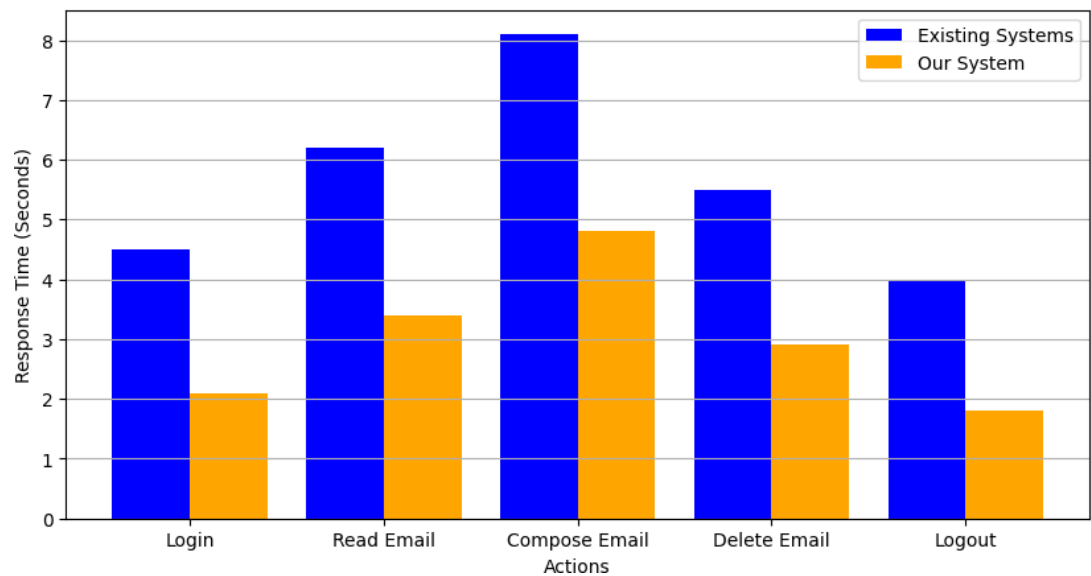


Fig.8 Response Time Comparison: Existing vs. Our System

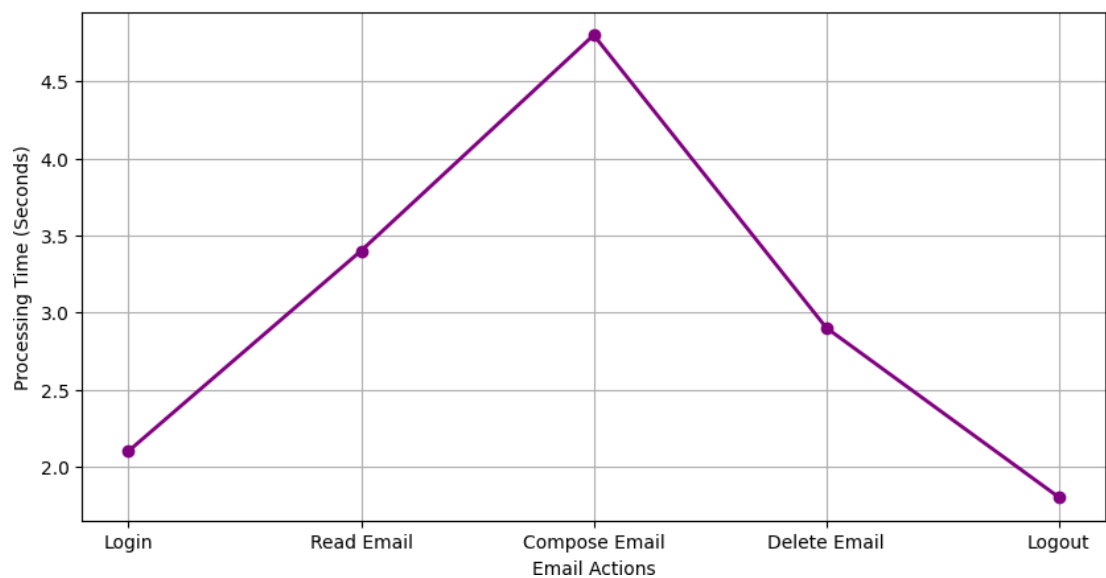


Fig.9 Real-Time Email Processing Time

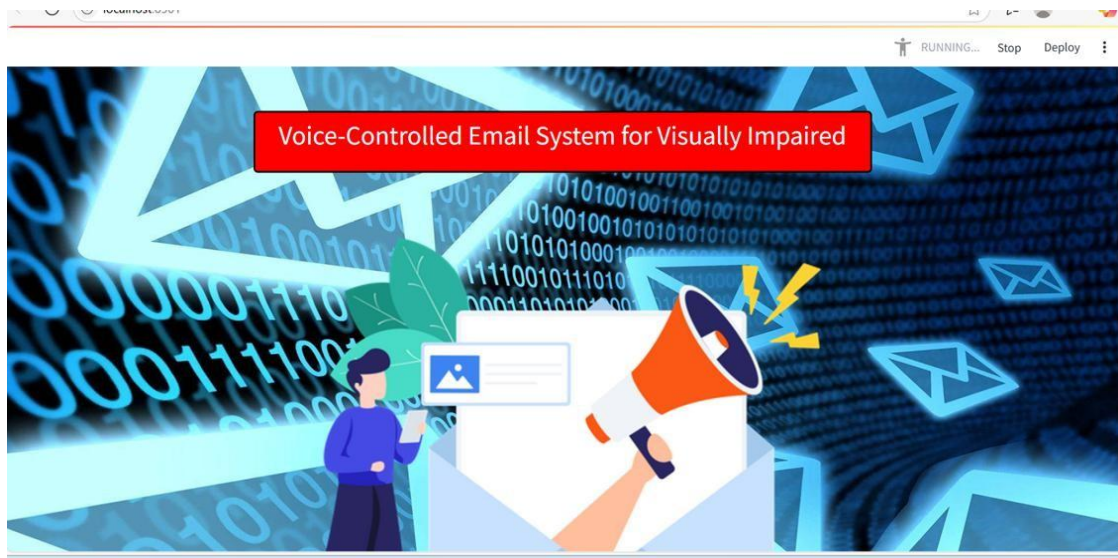


Fig.10 User Interface

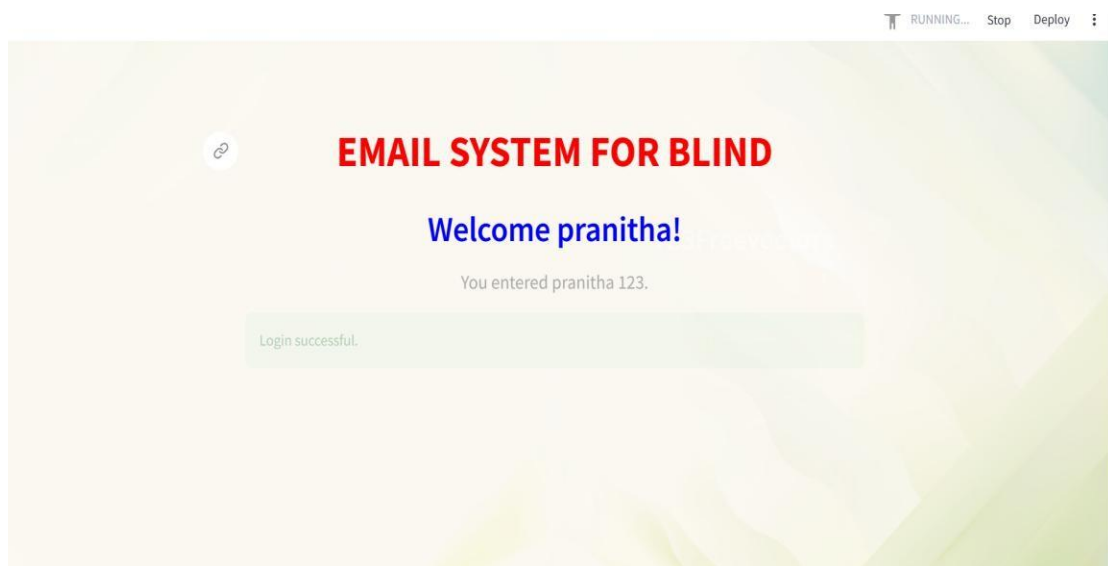


Fig.11 User Login Page

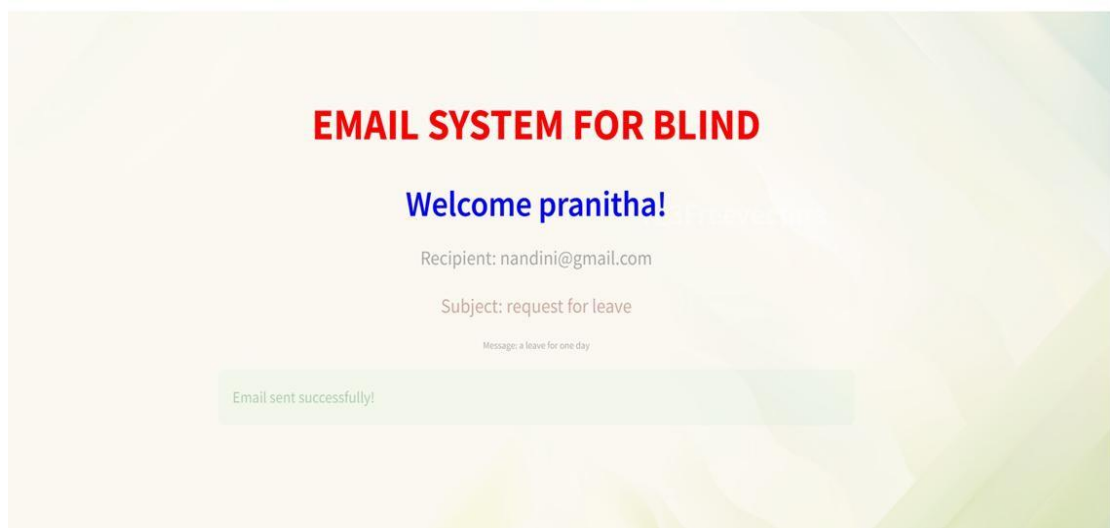


Fig.11 Sent Mail

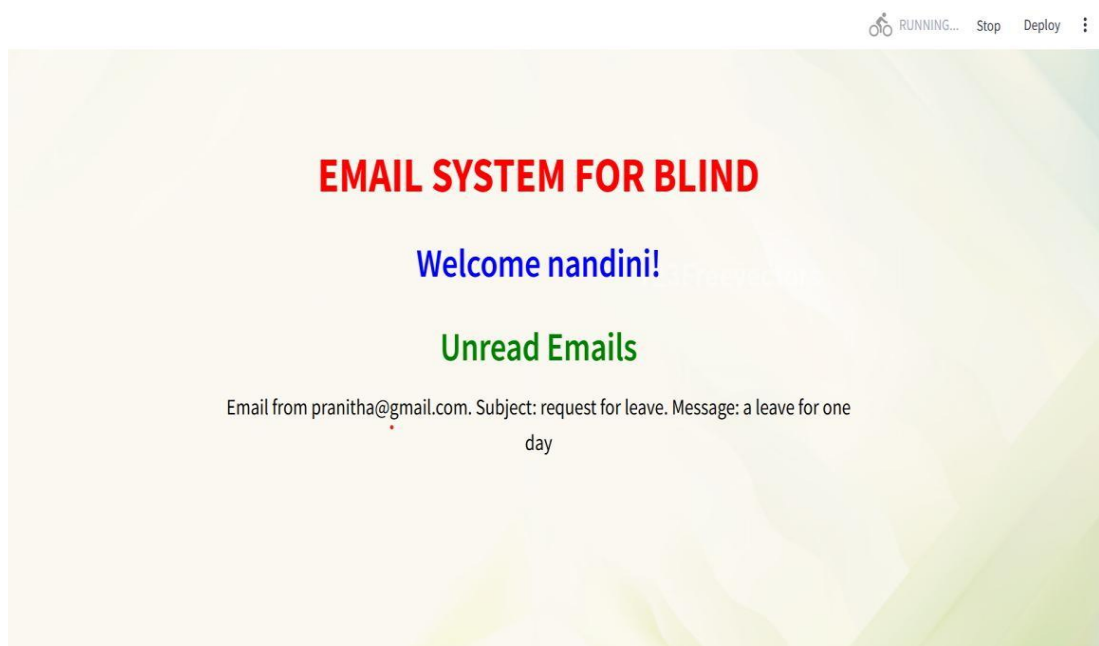


Fig.11 Received Mail

CHAPTER 7

Conclusion & Future Enhancements

59-61

7.1 Conclusion

7.2 Future Enhancements

7. CONCLUSION & FUTURE ENHANCEMENTS

7.1 Conclusion

In conclusion, the development of a voice-controlled email system tailored specifically for blind and visually impaired users marks a significant stride toward digital inclusivity and accessibility. This system effectively addresses the challenges posed by traditional email platforms, which are heavily reliant on graphical interfaces and visual cues, by offering a hands-free, voice-driven solution. Through the integration of speech recognition, natural language processing, and text-to-speech technologies, users can now independently log in, compose, send, receive, delete, and read emails without the need for external assistance. The system's ability to correct grammar using NLP ensures that composed messages are clear and professional, while TTS allows users to listen to incoming messages with ease. This end-to-end voice interaction offers users a new sense of autonomy and confidence in managing their digital communications.

Furthermore, the successful implementation and testing of the system have proven its practicality and effectiveness in real-world scenarios. The intuitive voice interface, combined with robust backend support via a custom SQL database and lightweight UI, ensures seamless user experiences. While the current system provides a solid foundation, future enhancements such as multilingual support, emotion recognition, and AI-based voice assistants could further personalize and improve the platform. This project not only fulfills a pressing societal need but also serves as a powerful example of how modern technology can be leveraged to create inclusive solutions that empower all users, regardless of physical limitations. In doing so, it contributes meaningfully to bridging the accessibility divide and advancing the goal of equal digital opportunity for everyone.

7.2 Future Enhancements

While the current system effectively empowers visually impaired individuals to manage email communication through voice commands, there is significant scope for further enhancement. One promising direction is the integration of multilingual support, enabling users to interact with the system in their native languages. This would broaden accessibility across diverse linguistic communities and make the system more user-friendly for non-English speakers. Additionally, incorporating AI-based voice assistants like Google Assistant or Alexa could elevate the user experience by making interactions more natural and responsive. These assistants could handle complex queries, manage conversations, and even learn from user behavior to improve over time. Another valuable addition would be offline functionality, where users can dictate and queue emails to be sent later when an internet connection is available, ensuring uninterrupted service in areas with limited connectivity.

In the future, the system could also incorporate emotion recognition and context-aware interactions to enhance personalization and user satisfaction. For example, recognizing stress or urgency in a user's voice could help the system prioritize certain emails or suggest appropriate responses. Security enhancements, such as voice biometrics for user authentication, would provide an extra layer of protection and ensure that email access remains confidential and secure. Integration with wearable devices like smartwatches or smart glasses could further improve accessibility by offering hands-free controls and mobility. Furthermore, continuous improvement of the speech recognition engine using deep learning models would increase accuracy, even in noisy environments or for users with different accents. These future enhancements would not only make the system more intelligent and efficient but also reinforce its mission of inclusivity, independence, and digital empowerment for all visually impaired individuals.

REFERENCES

- [1] Shah, Harsh D., Amit Sundas, and Shabnam Sharma. "Controlling email system using audio with speech recognition and text to speech." 2021 9th international conference on reliability, infocom technologies and optimization (trends and future directions)(ICRITO). IEEE, 2021.
- [2] Umbare, Mrs Rupali, et al. "Voice Controlled Email System Using Speech Recognition."
- [3] Umbare, Rupali, et al. "AI Based Email System using Speech Recognition."
- [4] Chimkode, Suresh, and Sara Fatima. "Voice Based E-Mail System For Blind People Using Speech Recognition Technology."
- [5] Chimkode, Suresh, and Sara Fatima. "Voice Based E-Mail System For Blind People Using Speech Recognition Technology."
- [6] Danish, Kunwar, et al. "Speech recognition based E-mail for visually impaired." AIP Conference Proceedings. Vol. 3075. No. 1. AIP Publishing, 2024.
- [7] Sharma, Aditi, et al. "An effective approach to speech-based email assistance for visually impaired people." 2022 8th International Conference on Signal Processing and Communication (ICSC). IEEE, 2022.
- [8] Kanyal, Hoshiyar Singh, et al. "Resume Designing using Speech Recognition System." 2021 10th International Conference on System Modeling & Advancement in Research Trends (SMART). IEEE, 2021.
- [9] Luo, Qi. "The improving effect of intelligent speech recognition System on english learning." Advances in Multimedia 2022.1 (2022): 2910859.
- [10] Abhishek, S., Harsha Sathish, and Arvind Kumar. "Aiding the visually impaired using artificial intelligence and speech recognition technology." 2022 4th International Conference on Inventive Research in Computing Applications (ICIRCA). IEEE, 2022.

APPENDIX - 1

PUBLICATION DETAILS

**International Conference on Recent Advancements in
Artificial Intelligence and Soft Computing
ICAISC-2025**
<https://methodist.edu.in/ICAISC-2025/>
Conference Dates : 5th - 6th June 2025



PAPER REVIEW FORM	
Paper ID	ICAISC2025-791
Paper Title	A Website for Email Sending for Blind People using Speech Recognition Technology
Review Status	Accepted with Modification
Consolidated Content review comments	• Paper should strictly formatted according to the standard format. • The paper could be revised with more detailed explanations to increase the understandability of the paper.

NOTE: Include all consolidated comments from the Reviewers in the Camera ready papers and make the paper to the satisfaction of the editor. Camera ready papers must confirm to the standard format. Failing this will result in rejection. Send your completed registration forms to icaisc2025@methodist.edu.in

CO – PO & PSO MAPPING

Course Outcomes: On successful completion of this Project, the students will be able to	
CO 1	Demonstrate a sound technical knowledge of their selected project topic
CO 2	Understand problem identification, formulation and solution
CO 3	Design engineering solutions to complex problems utilizing a systems approach
CO 4	Communicate with engineers and the community at large in written and oral form
CO 5	Demonstrate the knowledge, skills and attitudes of a professional engineer

Project Title	LEVERAGING AI IN RECRUITMENT: TRANSFORMING SCREENING, INTERVIEW AND SELECTION PROCESSES														
Course Outcomes (CO)	Programme Outcomes (PO) and Programme Specific Outcomes (PSO)														
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2	PSO 3
CO 1	3												3		3
CO 2	3	3	2										3	3	
CO 3	1		3	2	3	2	2				3	3	3	3	2
CO 4										3					
CO 5	2			2	2	2	2	2	3	3	3	3	3	2	3

Mapping of PO and CO

Influence of the course outcome (1 – Low, 2 – Medium, 3 – High)

RUBRICS FOR FINAL EXTERNAL EXAMINATION (FINAL ASSESSMENT)

Parameter/Marks(%)	Excellent (76%-100%)	Good (51%-75%)	Satisfactory (26%-50%)	Needs Improvement (0-25%)
Relevance & Innovation in Topic Selection (10M)	Topic is innovative, relevant, and well-justified with practical applications.	Topic is relevant but lacks strong novelty.	Topic is somewhat relevant but needs better justification.	Topic is not suitable or lacks clarity.
Analytical Thinking & Problem Interpretation (10M)	Demonstrates deep analytical thinking and strong logical interpretation of the problem.	Good analysis with minor refinements needed.	Basic analysis but lacks depth and synthesis.	Poor analysis with no clear synthesis of ideas.
Effective Problem-Solving & Implementation (10M)	Efficient problem-solving approach with optimized implementation.	Good problem-solving but with some inefficiencies.	Basic implementation, lacks optimization.	Poor problem-solving, incorrect or inefficient implementation.
Application of Engineering Techniques & Tools (10M)	Effective and appropriate use of modern tools, software, and technical methodologies.	Uses modern tools but needs better optimization.	Limited use of tools, lacks integration.	No or incorrect use of techniques and tools.
Societal Impact & Responsibility in Engineering (5M)	Clearly explains the societal benefits and responsible application of engineering solutions.	Considers impact but lacks depth.	Basic awareness of impact but not well integrated.	No awareness of how the project affects society.
Environmental Sustainability & Green Engineering (5M)	Strong integration of sustainability principles with minimal environmental impact.	Some environmental considerations but minor gaps.	Basic awareness, but not effectively applied.	No concern for environmental impact.

Parameter/Marks(%)	Excellent (76%-100%)	Good (51%-75%)	Satisfactory (26%-50%)	Needs Improvement (0-25%)
Ethical Considerations & Professional Responsibility (5M)	Strong ethical awareness in project work, decision-making, and implementation.	Ethical aspects considered but with minor gaps.	Some ethical considerations but not fully developed.	No ethical considerations in the project.
Self-Learning & Research Adaptability (5M)	Demonstrates strong ability for self-learning, exploring new domains, and independent research.	Shows interest in learning but relies partially on guidance.	Limited self-learning, depends mostly on given knowledge.	No effort towards learning beyond the syllabus.
Presentation Skills & Technical Communication (15M)	Excellent articulation, confidence, and clarity in explaining work.	Good communication, with minor areas for improvement.	Lacks confidence, some difficulty in conveying ideas.	Poor presentation, unable to explain the work.
Report Writing, Documentation & Formatting (15M)	Well-structured report with proper format, citations, clarity, and professional presentation.	Report is well-written but has minor formatting issues.	Report lacks depth, with missing or unclear sections.	Poorly written report, lacks structure and clarity.
Time, Budget & Resource Management (5M)	Well-planned execution, optimizing time and cost while managing resources effectively.	Good planning but minor inefficiencies in cost or time.	Basic planning but lacks detailed breakdowns.	Poor or no time and cost analysis.
Lifelong Learning & Future Scope of Work (5M)	Demonstrates commitment to continuous learning and identifies future research opportunities.	Shows some effort in exploring future possibilities but needs improvement.	Limited awareness of further learning or enhancements.	No effort toward learning or future research considerations.

Parameter	Maximum (Marks)
Relevance & Innovation in Topic Selection	10
Analytical Thinking & Problem Interpretation	10
Effective Problem-Solving & Implementation	10
Application of Engineering Techniques & Tools	10
Societal Impact & Responsibility in Engineering	5
Environmental Sustainability & Green Engineering	5
Ethical Considerations & Professional Responsibility	5
Self-Learning & Research Adaptability	5
Presentation Skills & Technical Communication	15
Report Writing, Documentation & Formatting	15
Time, Budget & Resource Management	5
Lifelong Learning & Future Scope of Work	5
Total Marks out of 100:	
Marks out of 60 Marks (Rounded-up)	

BIO – DATA OF TEAM MEMBERS

Personal Information

Name: Rayapati Chandini

Father's Name: Rayapati Khajavalli

Date of Birth: 11/03/2003

Gender: Female

Nationality: Indian

Marital Status: Single

Languages Known: English, Hindi, Telugu

Contact: +91-9441403295

Email: chandini.rayapati@gmail.com

Address: 42/1097-1-12, Kadapa, Andhra Pradesh, India

Educational Qualifications

Qualification	Institution	Board/University	Year	Percentage/CGPA
B.Tech (Computer Science)	KSRM College Of Engineering	JNTUA	2025	8.53 CGPA
Class XII	Narayana Olympiad College	Board of Intermediate Education A.P	2021	82%
Class X	Gurukul Vidyapeeth School	Board Of Secondary Education A.P	2019	98%

Technical Skills

- Programming Languages: C, Python, Java
- Web Technologies: HTML, CSS
- Tools: Git, VS Code

Projects

Title: A website for email sending for blind people using speech recognition technology

Description: Developed a website using speech recognition technology to send and receive email via speech.

Achievements / Certifications

- Certified in Python Programming – 2023
- Won 1st Prize in PPT presentation under CSI chapter – 2023

Declaration

I hereby declare that the above information is true to the best of my knowledge.

R.Chandini
(Student Signature)

Personal Information

Name: Palagiri Babjan

Father's Name: Palagiri Chinna Babaiah

Date of Birth: 20/04/2004

Gender: Male

Nationality: India

Marital Status: Single

Languages Known: English, Hindi, Telugu

Contact: +91-9381647466

Email: palagiribabjan143@gmail.com

Address: 2/39-A, vangimallavandlapalli, Sambepalli(M), Annamayya(dist), AP

Educational Qualifications

Qualification	Institution	Board/University	Year	Percentage/CGPA
B.Tech (Computer Science)	KSRM College Of Engineering	JNTUA	2025	8.3 CGPA
Class XII	Narayana Junior College	Board of Intermediate Education A.P	2021	72.3%
Class X	Sri Sai Concept School	Board Of Secondary Education A.P	2019	80%

Technical Skills

- Programming Languages: C, C++, Python, Java
- Web Technologies: HTML, CSS, Javascript
- Tools: Git, VS Code, MATLAB

Projects

Title: A website for email sending for blind people using speech recognition technology

Description: Developed a website using speech recognition technology to send and receive email via speech.

Achievements / Certifications

- Certified Python Programming – 2023
- Azure Fundamentals - Microsoft

Declaration

I hereby declare that the above information is true to the best of my knowledge.

P.Babjan
(Student Signature)

Personal Information

Name: Motakatla Charan

Father's Name: Motakatla Silas

Date of Birth: 29/06/2004

Gender: Male

Nationality: Indian

Marital Status: Single

Languages Known: English, Hindi, Telugu

Contact: +91-8008843082

Email: charanmotakatla99@gmail.com

Address: 2/78, duvvur mandal, machanapalle, Kadapa, Andhra Pradesh, India 516175

Educational Qualifications

Qualification	Institution	Board/University	Year	Percentage/CGPA
B.Tech (Computer Science)	KSRM College Of Engineering	JNTUA	2025	7.9 CGPA
Class XII	Sree Vidhya Junior ,Proddutur	Board of Intermediate Education A.P	2021	79%
Class X	Z.P.High School, Machanapalli	Board Of Secondary Education A.P	2019	97%

Technical Skills

- Programming Languages: C, C++, Python, Java
- Web Technologies: HTML, CSS, JavaScript
- Tools: Git, VS Code

Projects

Title: A website for email sending for blind people using speech recognition technology

Description: Developed a website using speech recognition technology to send and receive email via speech.

Certifications

- AWS Devops – BRAINOVISION
- 3D Gaming – WORKSHOP

Declaration

I hereby declare that the above information is true to the best of my knowledge.

M.Charan
(Student Signature)

Personal Information

Name: Peddeti Pranitha

Father's Name: Singaraiah Peddeti

Date of Birth: 01/06/2004

Gender: female

Nationality: Indian

Marital Status: Single

Languages Known: English, Hindi, telugu

Contact: +918919964227

Email: pranithapeddeti@gmail.com

Address: Devanagar(v), Attlur(m), Kadapa(d), Andhra Pradesh

Educational Qualifications

Qualification	Institution	Board/University	Year	Percentage/CGPA
B.Tech (Computer Science)	KSRM College Of Engineering	JNTUA	2025	5.3
Class XII	APSWR Balayogi Gurukulam Rayachoty(clg)	APSWR Gurukulam	2021	7.67
Class X	APSWR Balayogi Gurukulam Rayachoty(School)	APSWR Gurukulam	2019	7.3

Technical Skills

- Programming Languages: C, Python, Java
- Web Technologies: HTML, CSS, JavaScript
- Tools: Git, VS Code, MATLAB

Projects

Title: A website for email sending for blind people using speech recognition technology

Description: Developed a website using speech recognition technology to send and receive email via speech.

Achievements / Certifications

- Certified in Python Programming – 2023
- Won 1st Prize in Coding Hackathon – 2022

Declaration

I hereby declare that the above information is true to the best of my knowledge.

P.Pranitha
(Student Signature)